

人工智能基础与应用 · 大模型实训

人工智能基础与应用-总集

人工智能基础与应用 · 大模型实训

孙承泽 · 能动强基 2501 · 西安交通大学

2026-09-17 生成 · 版本 v1.0

Markdown 母版：人工智能基础与应用/讲义/ 下的 .md 文件 · 配套上游：CS336 A1-A5 · nanochat · minimind

目录

目录	2
人工智能基础与应用 I · 学科 README (课程轨 + 实训轨 双轨合一)	8
必读顺序 (新会话照此开局, 不许跳)	8
课程档案 (CORE100299, 2026-09-16 首课)	8
目录地图	8
课题单元形态	9
当前状态 (只改这一行, 细节进 memory)	9
一课两用 (本学科最大红利)	9
与其它学科联动	10
讲义库 · 使用方法 (三师制里的“我”那一腿)	11
讲义规范 (每讲必含 8 件)	11
📖 下载 PDF (离线阅读 / 打印用)	11
目录与状态	11
使用流程 (照做)	12
讲义 M0 · 数学与机器学习地基	13
○、本讲要回答的 5 个问题	13
一、自动微分: 为什么反向传播只要一遍	13
二、信息量、交叉熵、困惑度、bpb (跨分词器可比性的根源)	13
三、算力与显存记账 (估数量级是核心能力)	14
四、数值稳定性: softmax 与 logsumexp	14
五、“loss 不降”的排查顺序 (把知识变成动作)	15
六、能立刻跑的实验	15
七、代码级提问 (3-5 道)	15
八、记忆锚	16
讲义 M1 · 从 n-gram 到 Transformer	17
○、本讲要回答的 5 个问题	17
一、n-gram: 用计数做语言模型	17
二、词向量: 从 one-hot 到分布式表示	17
三、RNN / LSTM: 把“历史”压进一个向量	17
四、seq2seq 与注意力 (2014): 注意力的真正起源	18
五、常见误解	18
六、能立刻跑的实验	18
七、代码级提问 (3-5 道)	19
八、记忆锚	19
讲义 M2 · 分词与表示	20
○、本讲要回答的 5 个问题	20
一、压缩视角: BPE 到底在做什么	20
二、为什么必须字节起步 (以及代价)	20

三、训练算法：从 $O(n^2)$ 到官方要求的 <1.5 秒	21
四、编码：回放合并历史，而不是最长匹配	21
五、接口设计决定内存上界（encode vs encode_iterable）	21
六、词表大小的三方权衡（第 1、4 题的答案框架）	22
七、常见误解（每条配反例）	22
八、能立刻跑的实验（讲完就做，知行咬合点）	22
九、代码级提问（3-5 道，先写预测再动手）	23
十、记忆锚	23
十一、本讲与后续的接口（预告）	23
讲义 M3·Transformer 内部机制	24
○、本讲要回答的 5 个问题	24
一、注意力：加权检索 + 为什么缩放 $\frac{1}{\sqrt{d_k}}$	24
二、多头：拆开看不同关系，但总算力不变	24
三、位置编码：从“贴标签”到“转角度”	25
四、归一化与残差：让 100 层能训起来的两件套	25
五、FFN、参数量与 KV cache（把结构换算成资源）	26
六、常见误解（每条配反例）	26
七、能立刻跑的实验与改坏实验（课题 02 的里程碑）	26
八、代码级提问（3-5 道）	27
九、记忆锚	27
讲义 M4-M9·展开大纲（按周写完整版）	28
M4·训练：数据、优化、系统（W3-W9）	28
M5·推理与部署（W5-W8）	28
M6·后训练：从“会说话”到“会做事”（W10-W12）	28
M7·评测：把“感觉变好了”变成“确实变好了”（W9-W12）	29
M8·智能体：工具、记忆、harness、长程任务（W13-W14）	29
M9·前沿与安全（W15-W16 选题制）	29
展开时机表	30
知识地图·大语言模型全景（M0-M9）	31
判断“真的懂了”的三条硬标准（本学科通用）	31
M0·数学与机器学习地基	31
M1·语言模型史与前 Transformer 时代	31
M2·分词与表示	31
M3·Transformer 架构与内部机制	32
M4·训练：数据、优化、系统	32
M5·推理与部署	32
M6·后训练：从“会说话”到“会做事”	33
M7·评测：把“感觉变好了”变成“确实变好了”	33
M8·智能体：工具、记忆、harness、长程任务	34
M9·前沿与安全（持续跟进区，不是一次性学完）	34
模块 ↔ 课题 ↔ 周次 对照	34
与课程（CORE100299）六板块的映射	35

对标教学资源清单（2026-09-17 核实）	36
一、主线三件套（工程 + 判分，本学科骨架）	36
二、理论线的“对标名师”（补“讲”的深度）	36
三、评测与环境类（M7/M8 的判分件）	37
四、明确标注的坑（用前必读）	38
五、三师制使用方法（每个知识点的标准动作）	38
知行合一规程（本学科教学法核心）	39
一、把“知行合一”变成可执行的四件咬合	39
二、知识卡规范（每个知识点一张，落知识卡/或课目录）	39
三、讲课节拍（每周两个时段，各 1.5-3h）	39
四、代码级提问（“知”与“行”的咬合点，本学科独创）	40
五、“真的懂了”的三条硬标准（与《知识地图》一致，此处为验收动作）	40
六、反讲机制（费曼 + 红队）	40
七、与其它章程的关系	40
实训路线图·16 周 × 11 课题 × 唐杰 7 条 × CS336 映射	41
一、目标（一句话）	41
二、唐杰 7 条 ↔ 本学科课题 ↔ 判分来源（主映射表）	41
三、16 周课表（周次 / 课题 / 交付物 / GPU 需求）	41
四、11 课题总表（手写要点 / 判分来源 / 通过证据）	43
五、规模三档（零预算版）	44
六、跨学科搭桥（讲课时主动用）	44
造轮子边界·谁写代码（本学科最高契约）	45
一、三条铁律	45
二、手写清单（学习目标，共 10 项）	45
三、代写清单（脚手架，Agent 负责，共 8 项）	45
四、协作区（一起定，你拍板）	46
五、违规示例（判例，会长大）	46
六、边界自检（每次动手前问一遍）	46
AI 使用红线（本学科版）	47
一、最高原则	47
二、手写区：Agent 应当做（SHOULD）	47
三、手写区：Agent 禁止做（SHOULD NOT，照抄 CS336 并加严）	47
四、脚手架区：Agent 应当做（且必须做完）	47
五、例外与补偿（本学科特有，CS336 没有）	48
六、教学法提示（照抄 CS336 的方法论，本学科已内化）	48
七、学术诚信与“知行合一”的关系（本学科的立场）	48
八、越界举报（自检清单，每轮自问）	48
算力与成本预算（零预算版）	49
一、免费资源池（2026-09-17 核实）	49
二、三档规模（零预算下的能力边界）	49
三、成本估算（标记为估算，必须在课题 04 用实测替换）	49
四、每周配额分配（16 周）	50

五、硬纪律（出过事故的领域）	50
六、可选升级路径（仅在零预算实在卡住时启动，需用户拍板）	50
判分与验收标准	52
一、三层判分（每一层都必须留下原始输出）	52
二、通用红线（继承总纲 + 本学科加严）	52
三、每课题的“通过证据”清单（缺一不可）	52
四、成绩口径（对齐唐杰作业的 40% / 60%）	53
五、自欺自查清单（结项前必过）	53
上游锁定清单（本学科只读依赖）	54
一、骨架依赖（必须锁，判分与训练都靠它）	54
二、知识线依赖（只读，不入仓）	54
三、许可与合规注意	55
四、重审机制	55
课题 01 开题简报 · BPE 分词器	56
一、研究背景（这个问题历史上卡过谁）	56
二、研究问题（一句话，开放问题）	56
三、攻关线索（路标，不是答案）	56
四、里程碑（可勾选，3-6 个）	56
五、交付物要求	57
五之二、官方判分标准（2026-09-17 实跑探明，先知道标准再动手）	57
六、开题批（开场直接发给用户，3-5 题）	57
七、讲课锚点（知行合一的“讲”这一腿）	58
八、参考实现（只准用于对照，不准抄）	58
判卷记录 · 课题 01 BPE 分词器	59
L1 · 基线（2026-09-17 立科时实跑）	59
L2 · 导师审稿意见	59
L3 · 费曼收口	59
本课题的判分环境坑（已解决，记录备查）	59
课题 02 开题简报 · Transformer 骨架与训练循环	61
一、研究背景	61
二、研究问题（一句话）	61
三、攻关线索	61
四、里程碑	61
五、交付物	61
六、讲课锚点（讲义 M3 + 对标）	62
七、代码级提问示例（讲课中出，先预测后验证）	62
八、风险与提醒	62
课题 03 开题简报 · Scaling Law 预演与打脸链路	63
一、研究背景	63
二、研究问题（一句话）	63
三、攻关线索	63
四、里程碑	63

五、交付物	63
六、讲课锚点	63
七、代码级提问示例	64
八、风险与提醒	64
课题 04 开题简报·0.1B 端到端预训练（首次上机）	65
一、研究背景	65
二、研究问题（一句话）	65
三、攻关线索	65
四、里程碑	65
五、交付物	65
六、讲课锚点	66
七、代码级提问示例	66
八、风险与提醒	66
课题 05 开题简报·手写 Triton attention kernel	67
一、研究背景	67
二、研究问题（一句话）	67
三、攻关线索	67
四、里程碑	67
五、交付物	67
六、讲课锚点	68
七、代码级提问示例	68
八、风险与提醒	68
课题 06 开题简报·多卡与分布式训练	69
一、研究背景	69
二、研究问题	69
三、攻关线索	69
四、里程碑	69
五、交付物	69
六、讲课锚点	69
七、代码级提问示例	70
八、风险与提醒	70
课题 07 开题简报·数据清洗与去重	71
一、研究背景	71
二、研究问题	71
三、攻关线索	71
四、里程碑	71
五、交付物	71
六、讲课锚点	71
七、代码级提问示例	72
八、风险与提醒	72
课题 08 开题简报·SFT / DPO / RLVR 三方对照	73
一、研究背景	73

二、研究问题	73
三、攻关线索	73
四、里程碑	73
五、交付物	74
六、讲课锚点	74
七、代码级提问示例	74
八、风险与提醒	74
课题 09 开题简报·可验证环境 + harness	75
一、研究背景	75
二、研究问题	75
三、攻关线索	75
四、里程碑	75
五、交付物	75
六、讲课锚点	76
七、代码级提问示例	76
八、风险与提醒	76
课题 10 开题简报·长程 Agent 与 self-judge	77
一、研究背景	77
二、研究问题	77
三、攻关线索	77
四、里程碑	77
五、交付物	77
六、讲课锚点	78
七、代码级提问示例	78
八、风险与提醒	78
课题 11 开题简报·结项（NeurIPS 报告 + 现场 demo）	79
一、研究背景	79
二、研究问题（结项版）	79
三、攻关线索	79
四、里程碑	79
五、交付物	79
六、讲课锚点	79
七、自检清单（结项前必过，抄自《判分与验收标准》§ 五）	80
八、风险与提醒	80

人工智能基础与应用 I · 学科 README（课程轨 + 实训轨双轨合一）

2026-09-17 立（由“任务型轻工作台”升级为**双轨标准骨架**，升级条件由本 README 旧版预留）。本文件夹同时承载三件事：① **课程轨**——CORE100299 的课堂事务（考勤、随堂作业、大论文 70%）；② **实训轨**——把唐杰 2026 秋季作业清单（7 条）与 CS336 五份作业在**零预算**下走完的 11 个课题；③ **知识轨**——大语言模型 M0-M9 全景知识地图，知行合一。

两条轨共用一套 memory、一套证据规范、一套判分器——**同一份劳动产出两种成绩**（课程大论文 = 实训中期的技术报告）。

必读顺序（新会话照此开局，不许跳）

1. 仓库根 AGENTS.md（回合级 commit+push 是全仓库最高优先级）
2. _开局指令-给新会话 Agent.md（本学科唯一入口）
3. memory/HANDOFF.md（现在卡在哪、下一句该问什么）
4. 章程与地图/造轮子边界.md（★ 谁写代码——本学科最易出事故处）
5. 章程与地图/知行合一规程.md（★ 什么叫“学过”）
6. 其余按需：知识地图-大模型全景.md（学什么）/ 实训路线图.md（何时学）/ 对标教学资源清单.md（跟谁学）/ 算力与成本预算.md（花多少）/ 判分与验收标准.md（怎么算过）/ AI 使用红线.md（我能不能写）/ 上游锁定清单.md（依赖哪个版本）

课程档案（CORE100299，2026-09-16 首课）

- **推荐教材**：辛景民、武佳懿、郑南宁主编《人工智能概论》，清华大学出版社，2025
- **考核**：课程大论文 70% + 平时 30%（随堂作业 + 考勤）；**无期末笔试**
- **时间**：1-8 周，周三 9-10 节 + 周五 9-10 节，东 1 东-328（陈炜煌）
- **大纲六板块**：绪论 / 搜索求解 / 知识表示 / 机器学习 / 智能体 / 大语言模型
- 详见 资料原件/首课信息-20260916.md

目录地图

人工智能基础与应用/

└─ README.md	本文件（双轨导游图）
└─ _开局指令-给新会话Agent.md	唯一入口
└─ 章程与地图/	本学科“宪法层”
├─ 知识地图-大模型全景.md ☆	M0-M9：学什么（四件咬合：问题/对标/实验/证据）
├─ 对标教学资源清单.md ☆	跟谁学（22 项已核实资源 + 8 条坑）
├─ 知行合一规程.md ☆	什么叫学过（四件咬合 + 代码级提问 + 三条硬标准）
├─ 实训路线图.md	何时学（16 周 × 11 课题 × 唐杰7条 × CS336 + 知识讲解锚点）
├─ 造轮子边界.md ☆	谁写代码（手写 R1-R10 / 代写 B1-B8 / 协作 C1-C4）
├─ AI使用红线.md	双轨制下 Agent 的红线与例外
├─ 算力与成本预算.md ☆	零预算编排（Kaggle 30h/周 × 2×T4 + 实验卡制度 + 配额账本）
├─ 判分与验收标准.md	三层判分 + 每课题通过证据
├─ 上游锁定清单.md ☆	repo + commit + 许可 + 只用哪些路径

└─ 讲义/	☆私有 知识讲解主库 (三师制中"我"那一腿)
├─ pdf/	📖 PDF 成品: 全书 23 页 + 单讲分册 (下载即可读)
├─ README.md	使用方法 + 讲义规范 (8 件套) + 展开状态表 + PDF 下载
├─ M0-数学与机器学习地基.md	自动微分/熵与bpb/算力显存记账/数值稳定性/排查顺序
├─ M1-从n-gram到Transformer.md	n-gram/词向量/RNN/seq2seq+注意力/并行性
├─ M2-分词与表示.md	压缩视角/字节起步/算法复杂度/编码回放/接口与内存
├─ M3-Transformer内部机制.md	注意力与缩放/多头/GQA/RoPE/RMSNorm/SwiGLU/参数量/KV cache
├─ M4-M9-待展开大纲.md	M4-M9 骨架大纲 + 展开时机表
└─ 课程轨/	课堂事务 (本学科私有目录, 已登记)
├─ 课程进度.md	8 周 × 16 次课的进度锚点 + 作业/考勤台账
├─ 大论文/	选题 → 提纲 → 初稿 → 终稿 (70% 成绩)
└─ 资料原件/ 资料文本/	首课信息、活动留档、教材章节索引、讲义提取文本
└─ 环境/	☆私有 脚手架区 (已实跑验证)
├─ bootstrap_cpu.sh	T0 环境 (轻量档 9 秒; --with-torch 可选)
├─ 预检.md	会话门禁 (P100 一票否决) + 实验卡制度
├─ judge/	CS336 官方判分器接入 (test_*.py 未改一字) + tiktoken 离线词表
├─ 归档/make_run.py	证据归档: run.yaml (commit/预测/预算/产物哈希)
├─ 报告模板/	一页四段 (中文) + NeurIPS 骨架 (英文)
├─ pdf/	📖 讲义排版链: md2typst.py + 一键脚本 + 字体 (详见其 README)
└─ memory/	记忆系统六件 + drill-ledger
└─ 知识卡/	(可选, 量大时启用) 知识卡索引 → 见知行合一规程 §二
└─ 课题01-BPE分词器/ ... 课题11-结项/	作战单元 (**11 份开题简报已全部就位**)

私有目录登记 (依 仓库使用手册.md §二): 课程轨/, 环境/, 知识卡/, 以及各课题下的 手写/, 脚手架/, 证据/ 三个子目录, 均为本学科私有约定, 已在此登记。

课题单元形态

课题NN-名称/	
└─ 开题简报.md	背景 → 研究问题 → 攻关线索 → 里程碑 → 交付物
└─ 手写/	☆ 学习目标代码; Agent 只读禁写 (见 造轮子边界.md)
└─ 脚手架/	☆ Agent 全权负责: 编排/日志/绘图/判分/归档
└─ 证据/	☆ run.yaml + 日志 + 曲线 + 产物哈希 + 结论与反证 (no-go 不擦除)
└─ 报告.md	一页四段 (问题-动机-方法-结果)
└─ 判卷记录.md	自动判分结果 + 导师审稿 3-5 条 + 费曼收口记录

当前状态 (只改这一行, 细节进 memory)

● 2026-09-17 可开课, 教材已可下载: 三轨骨架 + 9 份章程 + 讲义库 (M0/M1/M2/M3 全文 + M4-M9 大纲)

· 11 份课题开题简报全部就位 + 判分器实跑接通 + 证据归档工具 + 讲义 PDF 全书 23 页 / 章程合订 28 页; 下一步 = 课题 01 开题 (BPE 分词器, 纯 CPU, 0 算力), 同步启动课程大论文选题。

📖 想读 PDF: 讲义/pdf/大模型实训讲义-全书.pdf (首选) · 章程与地图/pdf/章程与地图-全书.pdf

一课两用 (本学科最大红利)

课程要求	实训轨对应	复用方式
大论文 70% (第 8 周结课前)	课题 01-05 的成果	大论文 = 实训前五课题的中文技术报告 (选题可联动 turbine-blade-ai-platform)

课程要求	实训轨对应	复用方式
随堂作业 30%	每课题的判分记录 / 知识卡	直接取用, 零额外劳动
六板块 (搜索/知识表示/机器学习/智能体/LLM)	M0-M9 知识地图	见《知识地图》末尾的板块映射表

与其它学科联动

- probability/: RL 的期望/方差/条件期望地基已建成 → 讲 GRPO 时直接搭桥, 不重讲
- 论文精读/: Chinchilla / DPO / GRPO / ReAct / SWE-agent 等论文走论文精读流程
- turbine-blade-ai-platform (用户真实项目): 大论文与结项报告的应用案例来源

讲义库 · 使用方法（三师制里的“我”那一腿）

立库 2026-09-17。为什么要有这个库：用户指出“实操性够强，但知识讲解偏少”。《知识地图》回答“学什么”，《对标资源清单》回答“跟谁学”，本库回答“此刻这一讲到底讲了什么”——由我按你的水平写的主讲义。三者关系不是替代，是分工：

- 对标名师（Raschka / 李沐 / 李宏毅 / CS336）= 权威原始讲法，读它打地基
- 本库讲义 = 为你量身写的浓缩主干 + 与代码的对应 + 常见误解（省你时间）
- 我（对话里）= 只讲你卡住的那一层，出代码级提问，判你的理解

讲义规范（每讲必含 8 件）

#	部件	作用
1	本讲要回答的 3-5 个问题	带着问题读，读完能答才算过
2	主干讲解（≤150 字/概念块）	一次讲透，不挤牙膏
3	公式/伪代码骨架	这是“手写区”的地图， 不是答案
4	在代码里长什么样	指向手写/或上游 nanochat 的 函数与接口 （不给行号、不给实现）
5	常见误解（每条配一个反例）	抄自历届踩坑与官方测试的隐含要求
6	能立刻跑的实验	讲完就跑，知行咬合点
7	代码级提问（3-5 道）	改坏它/预测现象/给出诊断
8	记忆锚（隐喻 + 口诀 + 符号位置）	进 drill-ledger

📖 下载 PDF（离线阅读 / 打印用）

不用看屏幕读 Markdown——讲义已排成 PDF，直接下载：

下载这个	页数	说明
讲义/pdf/大模型实训讲义-全书.pdf	24	★ 首选：封面 + 版权说明 + 目录 + M0-M3 + 大纲
讲义/pdf/M0...M3.pdf 等单讲分册	2-5	手机上读、单讲打印、发给同学
章程与地图/pdf/章程与地图-全书.pdf	28	九份章程合订(学什么/跟谁学/什么叫学过/何时学/谁写代码)

排版与验收：正文思源宋体、标题黑体、代码等宽；全书 24 页里 99 条去重公式已**逐条单独编译验证**过，每讲章节也做了齐备性自检（防排版吞内容）。重生成与维护见 环境/pdf/README.md。

```
cd 人工智能基础与应用 && 环境/pdf/生成讲义PDF.sh # 每讲 + 全书
cd 人工智能基础与应用 && 环境/pdf/生成讲义PDF.sh --charter # 章程分册
```

铁律：母版是这里的 .md，PDF 是派生物。改了讲义必须重跑一次生成脚本。

目录与状态

讲义	模块	对应课题/周次	状态	PDF
M0-数学与机器学习地基.md	M0	贯穿, W1 起	✓ 已写	✓ 5 页
M1-从n-gram到Transformer.md	M1	课题 02, W2	✓ 已写	✓ 4 页
M2-分词与表示.md	M2	课题 01, W1 (当前)	✓ 已写	✓ 5 页
M3-Transformer内部机制.md	M3	课题 02, W2-W3	✓ 已写	✓ 5 页

讲义	模块	对应课题/周次	状态	PDF
M4-M9-待展开大纲.md	M4-M9	W4-W16	🕒 大纲已就位，按周展开	✓ 4 页

PDF 生成日期：2026-09-17（母版有改动就要重跑 环境/pdf/生成讲义 PDF.sh）。

展开节奏：**每周讲之前，我把当周讲义写完整**（不留空讲义占位）；本文件表格同步更新状态。

使用流程（照做）

- ① 预习：读本讲“要回答的问题”（2 分钟）
- ② 读对标材料对应章节（≤25 分钟，带着问题读）
- ③ 读本库讲义（≤15 分钟，看主干与“代码里长什么样”）
- ④ 合上讲义，用自己的话讲给我听 → 我专挑含糊处追问
- ⑤ 进手写区（R 类任务）→ 跑判分
- ⑥ 做“改坏实验”→ 先预测 → 再跑 → 修好
- ⑦ 我出代码级提问，你作答 → 进 drill-ledger 建卡

纪律：讲义里**不含可直接粘贴到手写/的实现**（见《造轮子边界.md》）。讲义给的是“坐标系 + 陷阱 + 判据”，代码必须你自己长出来。

讲义 M0 · 数学与机器学习地基

对应：贯穿全程，W1 起 | 前置讲义，不单独占周次，随用随补 对标材料：李沐 d2l ch2–ch5 + ch11（优化）· Karpathy micrograd 视频 · probability/ 课题 01–08（已建成，直接搭桥）为什么先讲这个：后面所有的“为什么训练崩了”“为什么显存不够”“为什么换分词器 loss 不能比”，答案都在这一讲里。它不是复习，是工具箱。

〇、本讲要回答的 5 个问题

1. 梯度是怎么穿过 100 层网络的？为什么反向传播只要一遍？
2. 交叉熵、困惑度、bpb 三者的关系是什么？为什么跨模型只能比 bpb？
3. 训练一个 N 参数的模型，显存和算力到底怎么估？
4. 为什么 softmax 要先减最大值？不这样做会怎样？
5. “loss 不降”时，我该按什么顺序怀疑？

一、自动微分：为什么反向传播只要一遍

计算图：把 $f(x_1, \dots, x_n)$ 表示为有向无环图，每个节点是一次基本运算。链式法则：若 $y = f(g(x))$ ，则 $\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$ 。

两种方向的差别（这是本讲的第一个关键直觉）：

模式	做法	成本	适合
前向模式	从输入往输出推导数	每个输入要一遍 → n 遍	输入少、输出多
反向模式	从输出往输入推导数	一遍拿到全部输入的偏导	输入多、输出少 ← 我们的情况

我们的情况是“几亿个参数（输入多）→ 一个标量 loss（输出少）”，所以反向模式（就是我们说的反向传播）是**首选**：1 次前向 + 1 次反向 = 全部梯度。

梯度累加的工程含义：反向时每个节点把上游梯度**相加**。这解释了两个常见现象：①同一个张量被用了两次（如权重共享），梯度会累加；②既然梯度是“累加”的，就能**梯度累积**（gradient accumulation）——多个小批次的梯度加起来等价于大批次，这是后来显存不够时的核心手段（课题 04 会用到）。

在代码里长什么样：

- 手写/里会有一个 Value 类（或等价的张量自动微分），核心是 `.backward()`：按**拓扑逆序**遍历，每个节点用局部导数乘上游梯度累加到父节点。
- 上游对照：nanocat/optim.py 里参数更新前的 `p.grad` 就是这一机制的产物；CS336 A1 的 `run_gradient_clipping` 则是在使用梯度（先要能算出来）。

二、信息量、交叉熵、困惑度、bpb（跨分词器可比性的根源）

信息量：一个概率为 p 的事件携带的信息是 $-\log p$ （nats，底为 e ；若底为 2 则单位是 bit）。

熵：随机变量的平均信息量 $H(p) = -\sum_x p(x) \log p(x)$ 。直觉：“平均每个符号有多难猜”。

交叉熵：用模型分布 q 去编码真实分布 p 的代价

$$H(p, q) = -\sum_x p(x) \log q(x)$$

语言模型里 p 是“真实下一个 token”， q 是模型输出——这正好就是训练 loss。

困惑度： $\text{PPL} = \exp(H(p, q))$ (nats 为底时)。直觉最有用的一句话：“模型平均在多少个等概率选项里犯迷糊。” $\text{PPL}=10$ 相当于“平均 10 选 1”。

bpb (bits per byte)：

$$\text{bpb} = \frac{H(p, q) [\text{nats}]}{\ln 2 \times \# \text{bytes per token}}$$

分母是“平均每个 token 对应多少字节”。

为什么必须用 bpb 跨模型比较：交叉熵的单位是“每 token”，而 token 的大小随分词器变化——词表越大 token 越“大”，同样文本的 token 数越少，单 token 交叉熵天然越高。除以“每 token 字节数”就把记账单位统一到“每个字节”上，于是与分词器无关。这也解释了 M2 讲义的结论：换词表后 loss 不可比，bpb 可比。

在代码里长什么样：nanochat/loss_eval.py 专门做 bpb（而不是 loss）的评估；nanochat/core_eval.py 则是另一类指标（CORE，由 22 个数据集的准确率合成）。你在课题 01 的压缩率表，其实是在预演 bpb 的分母——先感受“每 token 多少字节”。

三、算力与显存记账（估数量级是核心能力）

参数量（Transformer 主项，细节见 M3）：

- 每层注意力 $4d^2$ （Q/K/V/O 四个 $d \times d$ ），MLP 约 $8d^2$ （SwiGLU 的等价形式）→ 每层 $\approx 12d^2$
- 词嵌入 $V \cdot d$ （权重共享时 1 份）

算力（训练）：业界通用估算

$$\text{总 FLOPs} \approx 6 \times N \times D$$

其中 N 是非嵌入参数量， D 是训练 token 数。拆解：每个参数在前向贡献 2 FLOPs（乘+加），反向是前向的 2 倍 → 合计 6。注意力部分另有 $12Lhqt$ 项（序列长度相关，短序列时可忽略）。

Karpathy 在 nanochat 文档里给的就是这个公式，并特别注明“与 Chinchilla 略有 1% 差异，因为不计嵌入与 softmax”。

显存（训练）：粗算每参数

项	字节数
参数 (bf16)	2
梯度 (bf16)	2
Adam 一阶动量 (fp32)	4
Adam 二阶动量 (fp32)	4
小计（不含激活、不含 fp32 主权重）	$\approx 12-16$
激活（随 batch $\times$ 序列长度增长，可用 checkpointing 换）	视配置

练习式直觉：0.1B 参数 $\times 16$ 字节 $\approx 1.6\text{GB}$ （优化器+梯度+权重）；剩下的全是激活与临时张量——这就是为什么 16GB 的 T4 训 0.1B 要精打细算（课题 04 的真实约束）。

在代码里长什么样：nanochat/gpt.py 里有 estimate_flops() 与 num_scaling_params() 两个方法，正是上面两段公式的代码化。课题 03 的 scaling law 就是靠这两个函数把“跑多大”换算成“花多少算力”的。

四、数值稳定性：softmax 与 logsumexp

问题： $\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$ 。若某个 $x_i = 1000$ ， e^{1000} 直接溢出成 inf。

解法：分子分母同乘 e^{-m} ($m = \max_j x_j$):

$$\text{softmax}(x)_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$$

数值上完全等价，但最大指数变成 $e^0 = 1$ ，永不溢出。“减最大值”不是技巧，是恒等变形。

logsumexp 技巧: $\log \sum_j e^{x_j} = m + \log \sum_j e^{x_j - m}$ ，这是所有“用 log 概率做加法”的算法（交叉熵、CTC、HMM、flash attention 的 online softmax）的共同基础。

为什么这很重要：课题 05 的 **online softmax** 就是“分块计算时如何边算边保持数值稳定”，其思想内核与这里的减最大值完全一致——你在 M0 学的东西，会在 Triton kernel 里原样再出现一次。

五、“loss 不降”的排查顺序（把知识变成动作）

1. **数据**：label 与输入错位了吗？分词是否正常（看打印出来的样本）？
2. **初始化**：全零/全同初始化会让对称性无法打破；检查 std 与层数是否匹配
3. **学习率**：太大 → loss 震荡/NaN；太小 → 曲线是平的。先用小数据能否过拟合一条样本自检
4. **形状与 mask**：mask 是否在 softmax 之前施加？是否广播到正确维度？
5. **数值**：是否出现 inf/nan？打开梯度范数监控
6. **数值精度**：bf16 在小模型/小梯度上可能不够（T4 没有 bf16，课题 04 要用 fp16 + GradScaler）

这份顺序本身就是“知行合一”：它将成为课题 04 的排查清单，而不是一段可背诵的文字。

六、能立刻跑的实验

```
# ① 手算 bpb 的换算（先自己推，再看输出对不对）
环境/.venv/bin/python - <<'PY'
import math
# 假设某模型：每 token 交叉熵 2.0 nats，平均每个 token 对应 4 个字节
loss_nats, bytes_per_token = 2.0, 4.0
ppl = math.exp(loss_nats)
bpb = loss_nats / math.log(2) / bytes_per_token
print(f"loss={loss_nats} nats → PPL={ppl:.2f}, bpb={bpb:.3f}")
print("→ 再推一遍：若词表翻倍使 bytes/token 变成 6，bpb 会怎么变？loss 呢？")
PY

# ② 验证“减最大值”是恒等变形（含极端值）
环境/.venv/bin/python - <<'PY'
import numpy as np
x = np.array([0.0, 1000.0, 1000.0])
naive = np.exp(x) / np.exp(x).sum()      # 溢出
stable = np.exp(x - x.max()) / np.exp(x - x.max()).sum()
print("naive :", naive, " ← 溢出成 nan")
print("stable:", stable, " ← 正确")
PY
```

七、代码级提问（3–5 道）

1. **【推导】** 用 e^{-m} 的恒等变形，证明“减最大值”不改变 softmax 的输出（写出两步）。

2. 【估算】 一个 0.1B 参数模型用 Adam 训练 (bf16 参数 + fp32 优化器状态): ①权重+梯度+优化器状态占多少 GB? ②若 T4 只有 16GB, 留给激活的预算是多少?
 3. 【预测】 训练中把 loss 从 fp32 计算改成 fp16 计算: 训练会更快还是更慢? 会出现什么新故障? (提示: T4 无 bf16; GradScaler 是干什么的)
 4. 【联系】 probability/ 里学过的“期望”和这里的“交叉熵”是同一个东西的两种说法吗? 试着用期望的符号把交叉熵写出来。
 5. 【诊断】 曲线在 400 步突然 NaN。按本讲 § 五 的顺序, 你会先看哪三个量?
-

八、记忆锚

- 隐喻: 反向传播 = “责任回溯” ——loss 是最终事故, 每个参数按“它贡献了多少”分摊责任; 责任可以累加, 所以梯度能攒 (梯度累积)。
- 口诀: “一遍反向、六倍算力、十六字节、先减最大值” (反向传播一遍 / 6ND / 每参数约 16 字节 / softmax 稳定性)
- 符号位置法: 把 $\text{bpb} = \frac{H}{\ln 2 \cdot B}$ 写在卡片中央, 分子 H 是“每 token 的惊讶”, 分母 B 是“每 token 的字节”, 中间一条分数线 = “把单位从 token 换成 byte” ——这条线就是跨分词器比较的桥。

讲义 M1 · 从 n-gram 到 Transformer

对应课题：课题 02 (W2) 前半 | 模块：M1 | 预计 1 个时段 对标材料：李沐 d2l ch8–ch10 (RNN → 注意力) · CS336 lecture 01 (概览) · 《Attention Is All You Need》原文 (走论文精读) **为什么讲这段**：不理解 attention 之前的世界，就不知道它在解决什么问题——而“知道它在解决什么问题”，是你将来能判断“某个新架构值不值得看”的唯一依据。

〇、本讲要回答的 5 个问题

1. n-gram 语言模型为什么必然失败？它的失败方式是什么？
2. 困惑度 (perplexity) 为什么能当指标？它和 M0 的交叉熵什么关系？
3. RNN/LSTM 解决了什么，又为什么被淘汰？
4. seq2seq + attention (2014) 与 Transformer (2017) 的关键差别在哪？
5. 为什么说“并行性”是 Transformer 取胜的主因，而不是“注意力更聪明”？

一、n-gram：用计数做语言模型

链式法则： $P(w_1 \dots w_T) = \prod_t P(w_t | w_{<t})$ 。n-gram 的近似：只依赖前 $n-1$ 个词， $P(w_t | w_{t-n+1} \dots w_{t-1})$ ，用语料计数估计：

$$P(w_t | w_{t-1}) = \frac{c(w_{t-1}w_t)}{c(w_{t-1})}$$

两个死穴（这就是“为什么历史必然翻篇”）：

死穴	具体表现
稀疏	没见过的组合概率为 0 → 平滑 (add-k / 回退 / Kneser-Ney) 只是打补丁
上下文太短	$n=5$ 时也只能看 4 个词；而“这句话的主语是谁”可能要跨 50 个词

评价指标：困惑度 $PPL = \exp\left(-\frac{1}{T} \sum_t \log P(w_t | w_{<t})\right)$ —— 正是 M0 里交叉熵取指数。PPL 越低，模型对下一个词越不“惊讶”。这个指标一路沿用到今天 (bpb 是它的分词器无关版本)。

二、词向量：从 one-hot 到分布式表示

one-hot 的问题：任意两词的内积都是 0 —— “猫”和“狗”的距离与“猫”和“民主”一样远。语义无从谈起。

word2vec (skip-gram + 负采样)：用一个“给定中心词预测上下文词”的浅层网络，训练出的中间层权重就是词向量。结果是著名的向量算术： $\overrightarrow{king} - \overrightarrow{man} + \overrightarrow{woman} \approx \overrightarrow{queen}$ 。

关键转折：人们发现“预测下一个词”这个任务逼着模型学出语义结构。这正是后来 GPT 路线的思想雏形：语言建模本身就是最好的自监督任务。

三、RNN / LSTM：把“历史”压进一个向量

RNN: $h_t = \tanh(W_h h_{t-1} + W_x x_t)$ ，用 h_{t-1} 携带历史。LSTM：加门控（遗忘门、输入门、输出门）让信息可以“长期保留”。

为什么被淘汰（两个根因，都是工程性的）：

1. **梯度消失/爆炸**: BPTT 要对时间展开求导, 梯度是 $\prod_t \frac{\partial h_t}{\partial h_{t-1}}$ 的连乘。连乘里每个 Jacobian 的谱半径 $<1 \rightarrow$ 指数衰减; $>1 \rightarrow$ 爆炸。这是结构性问题, 不是调参能解决的。
2. **无法并行**: h_t 依赖 h_{t-1} , 必须按时间顺序串行计算。GPU 有上万核心却只能一次算一步 —— 训练吞吐被“时间”卡死。

这是本讲最重要的一句话: 深度学习的历史不是“模型更聪明”的历史, 而是“让更多算力能有效地作用于更多数据”的历史。Transformer 赢在能并行。

四、seq2seq 与注意力 (2014): 注意力的真正起源

seq2seq 结构: 一个 RNN 把源句编码成一个固定长度向量, 另一个 RNN 从它解码出目标句。

瓶颈: 整句信息被压进一个向量 —— 句子越长, 丢得越多。

Bahdanau 注意力 (2014) 的解法: 解码每一步时, 回头扫描编码器的所有隐状态, 算出“当前该关注哪几个位置”, 加权求和作为当前步的上下文。→ 这就是 **QKV 的雏形** (解码器状态=Query, 编码器隐状态=Key/Value)。

关键历史结论: 注意力最初是给 RNN 打补丁的, 不是 Transformer 的胜利品。Transformer 的贡献是: “既然注意力这么好用, 那还要 RNN 干什么?” —— 把 RNN 整个删掉, 只用注意力 + 前馈层, 于是整段序列可以并行计算。

路径长度对比 (理解“为什么更快”的关键):

架构	任意两位置的依赖路径	并行性
RNN	$O(L)$ (要穿过 L 步)	✗ 串行
Transformer	$O(1)$ (一步直达)	✓ 全并行

代价是注意力本身 $O(L^2)$ 的计算量 —— 这是“用算力换并行”的经典交易, 也是后来 flash attention、稀疏注意力、线性注意力等一整条研究线的起点。

五、常见误解

误解	更正
“注意力是 Transformer 发明的”	2014 年 Bahdanau 就用于 seq2seq; 2017 只是去掉 RNN 并改成 self-attention
“self-attention 与 attention 是两回事”	同一机制, 只是 Q/K/V 都来自同一序列
“RNN 不行是因为效果差”	在长依赖上确实差, 但致命的并行性才是被淘汰主因
“PPL 越低模型越强”	只在同分词器、同数据下可比 (见 M2)

六、能立刻跑的实验

```
# 用 n-gram 的视角感受“上下文长度”的代价: 统计 bigram 的稀疏程度
环境/.venv/bin/python - <<'PY'
import collections, re
text = open("上游/assignment1-basics/tests/fixtures/tinystories_sample.txt",
encoding="utf-8").read().lower()
words = re.findall(r"[a-z']+", text)
uni = collections.Counter(words)
```

```

bi = collections.Counter(zip(words, words[1:]))
print(f"词数={len(words)} 不同词={len(uni)} 不同的相邻词对={len(bi)}")
print(f"→ 光是一个小语料，二元组就已经接近词数的 {len(bi)/len(uni):.1f} 倍；三/四元组的稀疏会爆掉")
print(f"→ 换成长度更大的语料，这个比值会继续上升（稀疏问题的定量感受）")
PY

```

七、代码级提问（3–5 道）

1. 【推导】为什么 BPTT 的梯度是 Jacobian 连乘？写出两步的式子，并说明“谱半径 < 1 就衰减”的含义。
2. 【对比】用一句话说清：Transformer 相比 RNN，用掉了什么、换来了什么？
3. 【判断】如果有一天硬件变成“串行极快、并行极慢”，Transformer 还会是主流吗？说理由。
4. 【联系】d2l 里 attention 的示意图和 M3 讲的 QK^T 是一回事吗？把两者对应起来。
5. 【指标】你的 n-gram 模型 PPL=120，另一个模型 PPL=80，但用的是不同分词器。能说后者更强吗？

八、记忆锚

- 隐喻：RNN = “传话游戏”——一句话传到第 50 个人已经面目全非；注意力 = “所有人同时看着原始白板”——谁离得远都不影响看清原文。
- 口诀：“n-gram 死于稀疏、RNN 死于串行、注意力先治 RNN、Transformer 删掉串行”
- 符号位置法：把 $O(L)$ 与 $O(1)$ 并排写在卡片上——左边是“要走 L 步”，右边是“一步直达”；下面是代价 $O(L^2)$ ，提醒你注意力不是免费的。

讲义 M2 · 分词与表示

对应课题：课题 01 BPE 分词器 (W1) | 模块：M2 | 预计 2 个时段 (3–5 小时) 对标材料：CS336 A1 handout (规格权威)· 李沐 d2l ch14.6 子词嵌入 (中文)· HF LLM Course ch2 读法：先读下面的“要回答的问题”，再读对标材料，最后回来看本讲义主干。

〇、本讲要回答的 5 个问题

1. BPE 训练究竟在优化什么？为什么说它“不是词汇表，而是压缩器”？
2. 为什么必须从字节起步？代价是什么？
3. 训练算法为什么容易写成 $O(n^2)$ ？如何做到官方要求的 <1.5 秒？
4. 编码时为什么必须“回放合并历史”，而不能“最长匹配优先”？
5. 同一个模型，换分词器后 loss 不可比——那该用什么指标？

一、压缩视角：BPE 到底在做什么

一句话：给定词表预算 V ，BPE 反复执行“找出当前出现次数最多的相邻 token 对 $\rightarrow$ 合并成一个新 token”。每合并一次，语料的总 token 数减少一点，直到词表填满 V 。

形式化：把语料看成一串字节 $b_1 b_2 \dots b_n$ 。定义当前切分下相邻对 (x, y) 的频次 $c(x, y)$ 。每步选

$$(x^*, y^*) = \arg \max_{(x, y)} c(x, y)$$

把语料中所有该对合并为新 token xy ，并把 (x, y) 记入 merges 列表（顺序即“资历”）。

关键推论 A（压缩率）：合并的收益可以直接量化——每合并一处，总 token 数减 1。所以“BPE 训练过程”就是“贪心压缩过程”，merges 列表本质是一本压缩字典，且按收益从大到小排序。

关键推论 B（为什么 loss 不可跨分词器比较）：token 越少 $\rightarrow$ 每个 token 承载信息越多 $\rightarrow$ 单 token 的交叉熵必然更高。这是记账单位问题，不是模型好坏。所以跨分词器比较必须换成与切分无关的单位：bits-per-byte (bpb)。

这条是新课标里最重要的一条：nanochat 全程用 bpb 而不是 loss 来跨模型比较，课题 03 (scaling law) 和课题 08 (RL 对照) 都会回到这里。

二、为什么必须字节起步（以及代价）

从 Unicode 字符起步的三个具体麻烦：

麻烦	具体例子	后果
词表爆炸且长尾	仅 CJK 就有 9 万余字符，常用仅数千	大部分字符训练不充分，嵌入近乎随机
永远有未登录	新 emoji（每年新增）、生僻字、方言字	无兜底机制 $\rightarrow$ OOV，直接崩
失去字节级共享	“妈”与“吗”共享部首但完全不同字符	学不到字形/字节层面的结构

从字节起步：任何文本 $\rightarrow$ UTF-8 $\rightarrow$ 0–255 的字节序列 $\rightarrow$ 这 256 个 token 永远在词表里。所以字节级 BPE 永远不存在“无法表示的输入”，只可能被切得更碎。这就是“零风险”的含义。

代价（必须知道，因为它是第 1 题的答案关键）：

- 汉字 UTF-8 占 3 字节；英文常用词常被合并成 1 token，而中文常需 1.5–2 个 token 表示 1 个汉字（取决于词表大小与语料配比） $\rightarrow$ 中文的“每字成本”高于英文

· 序列变长 → 注意力计算量 ($O(L^2)$) 上升 → 中文推理成本更高

三、训练算法：从 $O(n^2)$ 到官方要求的 <1.5 秒

朴素想法的三个陷阱（都会导致超时或超内存）：

1. 每步重扫全语料统计相邻对 → $O(V \cdot n)$, $V=500$ 、 $n \approx$ 百万级就是灾难
2. 用字符串做合并 (`text.replace("ab", "ab_new")`) → 每次 $O(n)$ 拷贝 + 无法区分“新合并的 token”与“原始字符”
3. 把语料整体读入内存再展开成 token 列表 → 5MB 语料 + 1MB 配额的内存测试必挂

正确方向的三个要点（方向，不是实现）：

- 词频级统计而非字符级：同一种“词”（如 the）在语料里重复百万次，只需统计一次它的结构，再按词频加权
- 增量更新：一次合并只影响“包含该对的相邻位置”，其余的对频次不变 → 只需局部更新
- 优先队列/堆：快速取出当前最高频的对，而不是每步全量排序

官方参考实现 0.38 秒，“玩具实现”约 3 秒——这 8 倍差距就在上面三点里。

四、编码：回放合并历史，而不是最长匹配

编码时我们没有语料统计，只有 merges 列表（一个有序的“合并历史”）。正确做法：

把文本按 UTF-8 切成字节数组

loop:

找出当前序列中所有相邻对里，在 merges 中“排名最靠前”（资历最老）的那一对

合并它

直到没有任何相邻对出现在 merges 中

为什么不是“最长匹配优先”？因为训练时每一次合并都遵循“最老优先”的规则；若编码用另一套规则，同一个子串在不同上下文会被切成不同 token，与 tiktoken/GPT-2 的逐 id 对齐测试必然失败。BPE 的编码本质是回放训练时的决策序列。

特殊 token 是另一套优先级：它们不参与合并，且在编码时先于普通文本被匹配。注意两个官方测试点名的情况：重叠的特殊 token（如 a 与 ab 同时存在）与尾随换行 (`<|endoftext|>\n`) ——这类边界的处理顺序会直接决定对错。

五、接口设计决定内存上界（`encode` vs `encode_iterable`）

官方测试里有一个耐人寻味的设置：

方法	官方态度
<code>encode(text)</code>	被 xfail 标记：“encode 预期会超过 1MB 配额”——超了也原谅
<code>encode_iterable(iterable)</code>	用 RLIMIT_AS 硬限 1MB，读 5MB 语料——超了就失败

为什么这不是双标：接口的输入形态决定了它的内存下界。`encode(str)` 的输入本身就是一整段字符串，它已经在内存里了——上界由输入决定，实现者无能为力。`encode_iterable(iterable)` 的契约是流式：输入是迭代器，实现就必须逐块处理、边读边 yield。任何 `list(iterable)` / `"".join(iterable)` / 先聚合再编码的写法，都违背了接口语义。

这是软件工程里最漂亮的一课：接口不只是名字，它是**约束**。同一个算法，换个接口，内存上界就从“输入大小”降到“块大小”。这个思想在课题 05（KV cache）、课题 07（数据流式清洗）、课题 10（长程 Agent 的记忆压缩）都会反复出现。

六、词表大小的三方权衡（第 1、4 题的答案框架）

词表从 V 扩到 4V，同时影响三件事：

受影响项	方向	机理
嵌入/输出层参数量	↑ 约 4 倍	参数 $\propto V \cdot d$ （若权重共享则 1 份，否则 2 份）
同文本序列长度 L	↓	更多常见片段被合并 → token 数下降
每 token 的注意力计算	↓ 约 L^2 下降	注意力是 $O(L^2 \cdot d)$
罕见 token 的训练充分度	↓	长尾 token 出现次数少，嵌入学不好

净效果**不是单调的**：太小 → 序列太长、算力烧在长度上；太大 → 参数浪费 + 长尾欠训练。经验上 32k–128k 是常见区间（GPT-2 用 50257，现代模型多在 100k 量级，且对多语言友好）。

中文特例：词表若中文覆盖不足，一个汉字可能被拆成 3 个字节 token，**中文成本直接翻 2–3 倍**——这是国产模型普遍重视中文词表预算的原因。

七、常见误解（每条配反例）

误解	反例
“一个 token 就是一个词”	<code>_transformer</code> 可能是 1 个 token，妈 可能是 3 个 token（UTF-8 三字节）
“分词只是预处理，不影响模型能力”	词表决定序列长度 → 决定有效上下文与算力分配 → 直接影响能学到什么
“编码用最匹配更聪明”	会与训练决策不一致 → 与 tiktoken 逐 id 对齐测试直接失败
“词表越大越好”	长尾 token 训练不足 + 嵌入层参数浪费；存在经验最优
“decode 一定无损”	只有在 完整 utf-8 序列 上才无损；单 token 解码可能是半个字符 (🐉)

八、能立刻跑的实验（讲完就做，知行咬合点）

```
# ① 跑判分基线（现在应当是 27 failed + 1 xfailed, 全部因缺失手写文件）
环境/judge/judge.sh 01 --quick

# ② 看官方参考 merges 长什么样（前 20 条）——体会“资历顺序”的直觉
head -20 上游/assignment1-basics/tests/fixtures/train-bpe-reference-merges.txt

# ③ 亲眼看字节：一个汉字到底占几个 token（用官方 fixture 的 GPT-2 词表）
环境/.venv/bin/python - <<'PY'
import tiktoken, os
os.environ["TIKTOKEN_CACHE_DIR"] = "环境/tiktoken-cache"
enc = tiktoken.get_encoding("gpt2")
for s in ["hello world", "你好", "你好，世界！", "🐉"]:
    ids = enc.encode(s)
    print(f"{s!r:16} → {len(ids)} 个 token  {[enc.decode([i]) for i in ids]}")
print("→ 汉字/emoji 被切成多个字节 token 的现象，一眼可见")
PY
```


九、代码级提问（3-5 道，先写预测再动手）

1. **【改坏预测】** 把训练里“并列最高频时任意取一个”改成“取字节序最小的一对”，`test_train_bpe` 会过吗？`test_train_bpe_speed` 呢？**先写预测和理由。**
2. **【诊断】** 你的实现通过了往返一致性测试，但 `test_train_bpe` 的 merges 比对失败。请按顺序列出你会检查的 4 件事（提示方向：预处理、并列规则、特殊 token、词表初值）。
3. **【代价估算】** 同一句话，词表从 32k 换到 128k：①嵌入参数量变几倍 ②序列长度大致变几成 ③**bpb** 会变吗？为什么？
4. **【接口】** 若把 `encode_iterable` 写成“先 `list(iterable)` 再拼接”，哪个测试会挂？挂的形式是断言失败，还是进程被直接杀掉？（想清楚 `RLIMIT_AS` 的作用方式）
5. **【迁移】** 课题 05 要写 KV cache，它会遇到和“`encode` vs `encode_iterable`”同构的内存问题吗？说说你的类比。

十、记忆锚

- 隐喻：BPE = “贪心的拼音缩写机”——不停找最常一起出现的两段，把它们缩成一个新记号，缩到最后词表满了；字典按先来后到排资历。
- 口诀：“字节兜底、频次贪心、老手优先、流式省内存”（对应：字节起步 / 训练目标 / 编码回放 / 接口约束）
- 符号位置法： V （词表）在上，撑着两件事：下面是 参数量 $\propto V \cdot d$ ，右边是 序列长度 $L \downarrow$ ；三者写成一个三角形，随时能想起“扩词表 = 拿参数换长度”。

十一、本讲与后续的接口（预告）

- **M3:** 序列长度 L 怎么进入注意力计算量 ($O(L^2)$) ——词表选择的代价在这一讲兑现
- **M4:** bpb 作为跨规模比较单位——scaling law 的纵轴
- **M5:** `encode_iterable` 的流式思想 → KV cache 的“不重算历史”
- **课程板块:** 本讲直接对应课程大纲的“大语言模型”入门，也是大论文“方法”章的第一段素材

讲义 M3 · Transformer 内部机制

对应课题：课题 02 Transformer 骨架 (W2-W3) | 模块：M3 | 预计 2-3 个时段 对标材料：CS336 lecture 03-04 + A1 handout (模型部分) · rasbt 《LLMs-from-scratch》ch3-ch4 (附录含 GQA/MLA/MoE) 这是全学科最核心的一讲：唐杰作业①的“从零写 Transformer”就落在这里。

〇、本讲要回答的 5 个问题

1. 注意力到底在算什么？为什么缩放因子是 $\frac{1}{\sqrt{d_k}}$ ？
2. 为什么要多头？多头比单头强在哪、代价是什么？
3. RoPE 凭什么比可学习位置嵌入更能外推？
4. RMSNorm 凭什么能替代 LayerNorm？pre-norm 与 post-norm 差在哪？
5. 一个 d 宽、 L 层、词表 V 的 Transformer 有多少参数？KV cache 有多大？

一、注意力：加权检索 + 为什么缩放 $\frac{1}{\sqrt{d_k}}$

计算式 (先记住形状，再看含义)：

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

其中 $Q, K, V \in \mathbb{R}^{L \times d_k}$ ， M 是因果掩码 (下三角为 0、其余 $-\infty$)。

含义：每个位置发出一个查询 q ，去和所有位置的键 k 做内积 (相似度)，softmax 归一化成权重，再对值 v 做加权平均。它就是“按相似度检索并汇总”。

为什么除以 $\sqrt{d_k}$ ：设 q, k 各分量独立、均值 0 方差 1，则内积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的方差是 d_k (方差可加)。 d_k 越大，内积的取值范围越宽 $\rightarrow$ softmax 越容易“饱和” (一个位置拿走几乎全部权重) $\rightarrow$ 梯度趋近 0 $\rightarrow$ 训练不动。除以 $\sqrt{d_k}$ 把内积的方差重新拉回 1 量级。这是“训练稳定性”的第一课。

因果掩码：语言模型不能看未来。做法是在 softmax 之前给未来位置加 $-\infty$ (实现上常用 -10^9 之类的大负数，而不是 0) —— 因为在 softmax 之后置零会改变概率归一化 (等价于把已分配的概率丢掉，剩下的位置没有重新归一)。

在代码里长什么样：手写/里会有 `scaled_dot_product_attention` 与 `multihead_self_attention` (含 causal 与 RoPE 两个变体)；上游对照 `nanochat/gpt.py` 的 `CausalSelfAttention`。注意接口而非实现：CS336 的 `tests/adapters.py` 规定了你要暴露的函数签名。

二、多头：拆开看不同关系，但总算力不变

做法：把 d 维拆成 h 份、每份 $d_k = \frac{d}{h}$ ，各自独立做注意力，再拼回来过一个输出投影 W_O 。

关键理解 (常被误解)：多头不增加注意力的总计算量——单个头用满 d 维时， $QK^\top$ 是 $L^2 d$ ； h 个头各 $\frac{d}{h}$ 维合计仍是 $L^2 d$ 。多头的收益在于归纳偏置：不同的头可以在不同子空间里学不同的关系 (有的学“上一个词是谁”，有的学“括号配对”，有的学“句子边界”)，比单一头更灵活。

变体 (现代模型常用)：

变体	做法	动机
MQA	所有头共享一份 K/V	KV cache 减小 h 倍，但质量略降

变体	做法	动机
GQA	头分组，组内共享 K/V（如 8 组）	折中：cache 减小、质量几乎不掉 ← 现代主流
MLA	把 K/V 压成低维潜变量再解压	更激进，DeepSeek 系

为什么都在动 K/V：推理时的瓶颈是 KV cache 的显存（见 § 五），它随 层数 × 序列长度 × 头数 线性增长，与“生成速度”直接相关。这是“架构为推理服务”的典型例子。

三、位置编码：从“贴标签”到“转角度”

问题：注意力的内积是置换不变的——把 token 顺序打乱，结果不变。所以必须注入位置信息。

三代方案：

方案	做法	缺陷
可学习绝对嵌入	学一个 $L_{\max} \times d$ 的查表	不能外推：训练 2048，推理 4096 时后一半没学过
正弦绝对编码（原论文）	$\frac{\sin}{\cos}$ 固定频率	可外推性弱，实践中被 RoPE 取代
RoPE（旋转位置编码）	把 q, k 按二维子空间旋转角度 $\theta \cdot m$	内积只依赖相对距离 $m - n$ ；外推性好

RoPE 的核心恒等式（这是它存在的全部理由）：

$$\langle R_m q, R_n k \rangle = \langle q, R_{n-m} k \rangle$$

即：在内积里只出现“位置差”，绝对位置被“旋转”吸收掉了。推导要点：旋转矩阵是正交阵， $R_m^\top R_n = R_{n-m}$ 。

频率分配：第 i 对维度的角频率 $\theta_i = 10000^{-2\frac{i}{d}}$ ——低维（ i 小）转得快（管近距离），高维转得慢（管远距离）。“10000”这个底数是“能覆盖多长上下文”的旋钮，这就是所谓 YaRN / NTK 插值等长上下文外推技巧的入口。

在代码里长什么样：手写/ 的 rope 与前缀 rope 的多头注意力；CS336 有单独的 run_rope 测试点。

四、归一化与残差：让 100 层能训起来的两件套

LayerNorm：对每个 token 的 d 维向量做 $\frac{x-\mu}{\sqrt{\sigma^2+\epsilon}}$ ，再缩放和平移。RMSNorm：只做 $\frac{x}{\sqrt{\text{mean}(x^2)+\epsilon}}$ ，再乘一个可学缩放 g 。

- 省掉了减均值与平移：更便宜（少一次归约、少一组参数）
- 效果相当：真正起作用的是尺度不变性（分母归一化），而中心化贡献有限
- 现代模型（LLaMA/Qwen/GLM 系）默认 RMSNorm

残差连接： $x \leftarrow x + f(\text{norm}(x))$ （pre-norm）。它防的是梯度消失：反向传播时，加法的 Jacobian 是单位阵，于是梯度有一条恒等通路可以直达浅层。没有它，几十层的网络基本训不动。

pre-norm vs post-norm（原论文是 post-norm）：

	形式	性质
post-norm	$\text{norm}(x + f(x))$	原版；深了需要小心 warmup，容易发散
pre-norm	$x + f(\text{norm}(x))$	训练稳，是现代默认；代价是“深度”的表示能力稍弱

课题 02 会做“去掉残差”的改坏实验：你会亲眼看到 loss 从“平稳下降”变成“几乎不动”。

五、FFN、参数量与 KV cache（把结构换算成资源）

FFN 的演化：两层线性 + 激活。原版 $d_{ff} = 4d$ 配 ReLU。SwiGLU 用三个矩阵：Swish(xW_1) $\odot$ (xW_3) 再过 W_2 ；为保持参数量不变，取 $d_{ff} \approx \frac{8}{3}d$ （3 个矩阵 $\times 8/3 \approx 4$ 个矩阵 $\times d$ 的等价参数量）。

单层参数量（ d 宽，标准 MHA）：

- 注意力： W_Q, W_K, W_V, W_O 各 $d \times d \rightarrow 4d^2$
- MLP (SwiGLU, $d_{ff} = \frac{8}{3}d$)： $3 \cdot d \cdot \frac{8}{3}d = 8d^2$
- 两处 RMSNorm 缩放： $2d$ （可忽略）
- 合计 $\approx 12d^2$ 每层 $\rightarrow$ 全模型 $\approx 12 L d^2 + Vd$ （权重共享时）

验算习惯：拿到任何模型的配置（如 $d=768, L=12, V=50257$ ），先心算 $12 \times 12 \times 768^2 \approx 85M$ ，加嵌入 $50257 \times 768 \approx 39M$ （共享则算 1 份） $\approx 124M$ —— 与 GPT-2 的 124M 吻合。这种“能不能心算”是 M3 的验收标志之一。

KV cache 显存（推理，重点）：

$$\text{bytes} = 2 \times L_{\text{layers}} \times L_{\text{seq}} \times n_{kv} \times d_k \times \text{bytes/elem}$$

（乘 2 是因为 K 和 V）。用 GQA 后 n_{kv} 从 h 降到组数，cache 直接按比例缩小——这就是 GQA 流行的真正原因。课题 05 会实测这个数字。

六、常见误解（每条配反例）

误解	反例 / 更正
“注意力=可解释的检索”	注意力权重高 $\neq$ 因果重要（已有大量反例）；它是 加权平均的权重 ，别过度解读
“多头=多算”	总 FLOPs 不变（§ 二），只是分成子空间
“位置编码只是加个向量”	RoPE 是 旋转 Q/K ，不是加到输入上；且它只影响内积的相位
“RMSNorm 是 LayerNorm 的近似”	它 省略了中心化 ，不是近似；在很多模型上效果更好且更省
“参数量大就一定强”	同样算力下，参数与数据要按 scaling law 配比（课题 03 的正题）
“因果 mask 随便加个大负数就行”	必须在 softmax 之前 ；加在之后会破坏归一化

七、能立刻跑的实验与改坏实验（课题 02 的里程碑）

基础验证（跑通即达里程碑 M1-M3）：

```
bash 环境/bootstrap_cpu.sh --with-torch # 课题02 起需要 torch
环境/judge/judge.sh 02 --quick # 形状与数值测试（判分器按需接入）
```

三组改坏实验（先写预测，再跑，再记录——这是本讲的核心作业）：

#	改哪里	先预测什么	预期观察（跑完记得回来对照）
1	去掉因果 mask(让所有位置互相可见)	训练 loss 会大幅下降，但采样出来的文本“语法很好、内容胡说”	典型“作弊”：loss 好看，能力为零
2	RoPE 换成可学习绝对嵌入	训练集内正常，超出训练长度立刻崩	外推性丧失；正是课程要“用实验看见”的性质
3	去掉残差连接	浅层（ ≤ 4 层）还能训，深了就完全不收敛	梯度消失的实证

记录要求：每条都要有“预测 $\rightarrow$ 现象 $\rightarrow$ 解释”三段，落进 课题 02/证据/。预测错是资产，不是污点。

八、代码级提问 (3–5 道)

1. 【推导】用“方差可加”说明： $q \cdot k$ 的方差是 d_k ，并解释为何要除以 $\sqrt{d_k}$ 而不是 d_k 。
2. 【心算】配置 $d = 1024, L = 24, V = 100000$ ，权重共享。估算参数量（结果用 G/M 表示），并拆出嵌入层占比。
3. 【改坏预测】把 causal mask 从“softmax 之前”挪到“softmax 之后”（对已归一化权重置零），训练会发生什么？给出两个可能的现象并说明理由。
4. 【联系】用 M0 的 logsumexp 解释：为什么 flash attention 的分块计算能保持数值稳定？
5. 【代价】序列长度从 2k 增到 32k：①注意力计算量变几倍 ②KV cache 变几倍 ③哪个先撞墙？

九、记忆锚

- 隐喻：注意力 = “会议室里的自由讨论”——每人 (query) 挑自己关心的话题 (key)，按关心程度 (softmax 权重) 听别人 (value) 说话；因果 mask = “不许听后面发言的人”；多头 = 同时开 h 个并行分会场；RoPE = 每个人的座位号刻在朝向上，所以只关心“相隔几个座位”。
- 口诀：“缩放防饱和、掩码在软前、多头不减算、旋转只认距、残差是命脉、归一挑 RMS”
- 符号位置法： $\frac{QK^T}{\sqrt{d_k}}$ —— 把 $\sqrt{d_k}$ 写在分数线下方，心里念“维度越高，内积越炸，除以根号拉回来”。

讲义 M4-M9 • 展开大纲（按周写完整版）

说明：本文件是 M4-M9 的**骨架大纲**（核心问题 + 关键概念 + 必读 + 实验 + 坑），每周讲课前我会把当周讲义**展开成完整版**（新增 M4-训练.md 等文件），不留空讲义占位。这样做的理由：这些模块严重依赖课题 04 起的**真实实验数据**，提前写死会出现“讲义与实测不符”。

M4 • 训练：数据、优化、系统（W3-W9）

要回答的问题：为什么同一个模型有人训得动有人训崩？scaling law 怎么指导算力分配？多卡为什么不是线性加速？

关键概念：预训练目标与数据配比 | 过滤与去重 | AdamW（解耦权重衰减的意义）| Muon | LR schedule（warmup + cosine）| 梯度裁剪 | 混合精度（bf16 vs fp16 vs fp32, GradScaler 何时必需）| 梯度累积 | DDP / FSDP / TP / PP | Chinchilla 的 compute-optimal | `--target_flops` 与“固定算力预算下的最优 (N, D)”

必读：CS336 lecture 05-11 + A2/A3 handout · nanochat/optim.py、base_train.py、runs/scaling_laws.sh · d2l ch11-ch12

实验：手写 AdamW 与调度 → 单卡固定算力下的 depth sweep（d4/d6/d8/d10）→ 拟合幂律 → **先写死外推预测** → 跑 d12 验证（打脸链路）

坑：把“loss 不降”当数据问题（常是 LR/初始化/精度）| 把 DDP 当免费加速（PCIe 下扩展效率可能只有 1.3-1.5×）| T4 无 bf16（必须 fp16 + GradScaler）| 显存不足先怀疑 batch 与精度，再怀疑硬件

M5 • 推理与部署（W5-W8）

要回答的问题：为什么生成必须逐 token？KV cache 省了什么、代价是什么？量化掉多少点？吞吐与延迟的取舍？

关键概念：自回归解码 | **KV cache**（显存公式见 M3 § 五）| continuous batching | PagedAttention | 采样策略（temperature / top-p / top-k / repetition penalty）| 量化（int8/int4/GPTQ/AWQ）| 投机解码 | 每 token 成本

必读：nanochat/engine.py（KV cache 实现）+ infer_bench.py · CS336 lecture 10 · rasbt《Reasoning》ch2

实验：从零写带 KV cache 的生成 → 测有/无 cache 的耗时与显存 → 贪心 vs top-p 对照 → batch size 对吞吐的影响

坑：以为“慢是因为模型大”（常是没批处理/没 cache）| 以为量化无损 | 忽略 prefill 与 decode 两个阶段特性完全不同（前者算力密集，后者显存带宽密集）

M6 • 后训练：从“会说话”到“会做事”（W10-W12）

要回答的问题：SFT / DPO / RLVR 各补什么能力？为什么 RLVR 提数学/代码却可能伤通用能力？reward hacking 长什么样？

关键概念：指令微调与 chat template | **损失掩码**（只算 answer 部分）| 奖励模型 | PPO 的四个模型 | DPO 的隐式奖励（推导：为什么可以不要 reward model）| **GRPO 的组内归一化优势** | KL 正则 | 拒绝采样 | 蒸馏与合成数据 | 过度优化（over-optimization）

必读: CS336 A5 (含 tests/test_grpo.py、GSPO/MaxRL) · rasbt《Reasoning from Scratch》ch3–ch8 · Nathan Lambert《RLHF Book》(免费在线, 17 章) · minimind/trainer/train_grpo.py · nano-aha-moment (单文件)

实验: 同基座做 SFT / DPO / GRPO 三变体 → 同一评测口径对照 → 手写 GRPO 损失并逐项注释 → **制造一次 reward hacking 并记录**

坑: 以为 DPO “更简单所以更好” | 以为 RL 一定比 SFT 强 | **把训练 reward 上升当能力提升** (不做 held-out 评测就是自欺) | KL 系数设错导致模型 “复读训练集答案”

M7 · 评测：把“感觉变好了”变成“确实变好了” (W9–W12)

要回答的问题: benchmark 分数可信吗？污染怎么判断？为什么必须固定口径？方差有多大？

关键概念: bpb / PPL / CORE | 下游基准 (GSM8K / MATH-500 / MMLU / HumanEval) | pass@k | 评测污染 | few-shot 与 prompt 敏感性 | 方差与误差棒 | LLM-as-judge 的偏置 | 可验证奖励 vs 学习型奖励

必读: rasbt ch3 (先建评测再上技术——全书教学法基石) · nanochat/core_eval.py (单文件 CORE) · CS336 lecture 12 · Berkeley Agentic AI MOOC 的 “Agent Evaluation” 讲

实验: 写一个数学题判定器 (解析 + 数值比对 + 容错) → 测它在 200 题上的准确率与**假阳性** → 给两个模型跑同一套评测并给误差棒

坑: 只看一个 benchmark 下结论 | 忽略 prompt 格式造成的巨大差异 | 把测试集当验证集反复调 | 只报均值不报方差

M8 · 智能体：工具、记忆、harness、长程任务 (W13–W14)

要回答的问题: 从“回答问题”到“完成任务”缺什么？为什么 harness 比模型本身更决定成败？self-judge 何时有效、何时自欺？

关键概念: 工具/函数调用 | ReAct 循环 (Thought-Action-Observation) | **context engineering** (李宏毅与 Karpathy 都专讲) | 记忆与压缩 | harness 设计 (沙箱、权限、反馈信号) | 长程任务的误差累积 | 多智能体 | self-judge / self-refine 的边界 | 可验证环境

必读: 李宏毅《Context Engineering》专讲 · Berkeley LLM Agents / Agentic AI MOOC (含 “Post-Training Verifiable Agents”) · HF Agents Course (三框架 + 公开排行榜) · rasbt ch5 (self-refinement) · PrimeIntellect-ai/verifiers · terminal-bench

实验: 手写最小 ReAct 循环 (不用框架) → 给两个工具 (计算器 + 代码执行) → 记录失败轨迹并分类 → 加 self-judge 做消融

坑: 以为瓶颈在模型智力 (常在你的工具接口与反馈信号) | 以为 self-judge 免费提升 (**它会放大原有错误**) | 上下文不做压缩导致长任务后段 “忘记目标”

M9 · 前沿与安全 (W15–W16 选题制)

要回答的问题: MoE / 长上下文 / 多模态改变了什么？对齐与安全是 “加一层” 还是 “改训练目标”？“AI 训 AI” 可行到什么程度？

关键概念: MoE 稀疏激活 (总参数 vs 激活参数) | 长上下文外推 | 多模态对齐 | 可解释性 | 红队与越狱 | 幻觉的成因 | 合成数据闭环 | self-play | 灾难性遗忘 | 模型合并

必读: CS336 lecture 04 (MoE) · opendilab/awesome-RLVR · LeapLabTHU/limit-of-RLVR · 论文精读清单联动

实验（三选一）：合成数据自训练一轮 / 拒绝采样提升数学 / self-play 生成题目 → 做最小闭环 + **明确写出这个方向的 no-go**

坑：把“前沿”当必须立刻掌握（正确姿势是**保持索引能力 + 每季度更新一次本模块**）| 只看结论不看评测口径 | 把“能跑通”当成“有结论”

展开时机表

讲义文件	需要先有	预计展开时间
M4-训练.md	课题 02 收口（能算 FLOPs/显存）	W3
M5-推理.md	课题 04 首轮训练成功（有模型可推理）	W5
M7-评测.md	有第一个模型输出	W9
M6-后训练.md	课题 07 数据管线 + 评测口径固定	W10
M8-智能体.md	有 SFT 后的模型	W13
M9-前沿.md	全部证据齐备	W15

知识地图 · 大语言模型全景（M0-M9）

立图 2026-09-17。本文件回答一个问题：要把大语言模型“彻底搞懂”，到底要懂哪些东西。每个模块都是“四个咬合件”：核心问题 → 对标材料 → 亲手实验 → 判分证据。只讲不做 = 没学（见《知行合一规程.md》）；只做不讲 = 学歪。

用法：模块不是线性章节，但顺序有依赖（M0→M1→M2→M3→M4→M5→M6→M7→M8→M9）。每周讲哪个模块，见《实训路线图.md》的“知识讲解锚点”列。

判断“真的懂了”的三条硬标准（本学科通用）

1. 能讲清：不用术语、用大白话讲给外行听，且能回答“为什么不是别的做法”。
2. 能指出代码位置：说出这个概念在本仓手写/或上游 nanochat/CS336 里具体是哪个函数、哪几行。
3. 能改坏并修好：故意改坏（去掉 mask、换掉归一化、打乱位置编码）→ 预测现象 → 跑实验验证 → 修回。（第 3 条是“知”与“行”的分界线；只会背定义的人在第三步一定翻车。）

M0 · 数学与机器学习地基

- 核心问题：梯度到底怎么传过 100 层？为什么归一化能让训练稳下来？交叉熵、困惑度、bpb 三者的关系是什么？参数量/FLOPs/显存怎么估？
- 关键概念：链式法则与计算图、雅可比、损失函数（MSE/交叉熵）、梯度下降族、正则化、数值稳定性、信息量与熵、复杂度记账（6ND 公式）
- 对标：李沐《动手学深度学习》ch2-ch5、ch11（优化算法）；Karpathy micrograd 视频；概率论课题 01-08（已建成，直接搭桥）
- 亲手实验：手写 micrograd（标量自动微分，200 行内）→ 手写 MLP 在 MNIST 子集上训到 >95%
- 判分证据：与 PyTorch autograd 的数值梯度对照表（误差 < 1e-6）+ 参数量/FLOPs 估算脚本实测偏差
- 常见误解：把 loss 当“正确率”；以为“梯度消失”是 bug 而不是结构性质；以为 d2l 的 MXNet 代码不用改

M1 · 语言模型史与前 Transformer 时代

- 核心问题：Transformer 解决了什么前人的痛点？为什么 attention 取代了 RNN？“语言模型”这个词 20 年里的含义变化？
- 关键概念：n-gram 与平滑、词袋、word2vec/GloVe、RNN/LSTM/GRU、seq2seq + 注意力、Bahdanau/Luong attention、并行化的动机
- 对标：李沐 d2l ch8-ch10（RNN → 注意力）；CS336 lecture 01（概览与分词）；《Attention Is All You Need》原文（入论文精读）
- 亲手实验：手写 bigram/trigram 语言模型算困惑度 → 手写一个最小 RNN LM → 对比同参数量的单层 Transformer
- 判分证据：三者在同一语料上的 bpb 对照表 + “RNN 为什么慢”的显式计时（逐步 vs 并行）
- 常见误解：以为“注意力=Transformer”；以为 attention 是 2017 才发明的（其实 2014 就有）

M2 · 分词与表示

- 核心问题：为什么不用字符？为什么不用整词？词表大小怎么权衡？中文/代码/数学符号怎么处理？

- **关键概念**: BPE (训练循环、merge 规则、特殊 token、bytes 回退)、WordPiece、Unigram、词表大小 $\leftrightarrow$ 序列长度 $\leftrightarrow$ 参数量的三方权衡、压缩率
- **对标**: CS336 A1 tokenizer 部分; nanochat tokenizer.py; HF LLM Course ch2 (“Implement BPE/WordPiece/Unigram from the ground up”)
- **亲手实验**: 手写 BPE (训练 + 编码 + 流式编解码往返) $\rightarrow$ 在 TinyStories 与中文语料上各训一个词表 $\rightarrow$ 测压缩率
- **判分证据**: tests/test_tokenizer.py、test_train_bpe.py 全绿 + 压缩率表 (中英文分别) + 往返一致性
- **常见误解**: 以为 token 就是“词”; 忽略中文一个汉字可能被切成 2-3 个 byte token 导致实际序列变长

M3 • Transformer 架构与内部机制

- **核心问题**: attention 到底在算什么? 为什么需要多头? 位置编码为什么从正弦换成 RoPE? RMSNorm 凭什么比 LayerNorm 更常用? 残差在防什么?
- **关键概念**: QKV 与缩放点积、因果 mask、多头与 GQA/MQA、RoPE (含外推)、RMSNorm、pre-norm vs post-norm、FFN/SwiGLU、权重共享、初始化与 QK-norm、参数量分解
- **对标**: CS336 lecture 03-04 + A1 model 部分; rasbt 《LLMs-from-scratch》ch3-ch4 (含 MoE/GQA/MLA/SWA 附录); 李宏毅 GenAI-ML; nanochat gpt.py
- **亲手实验**: 手写完整 Transformer 层 (含 RoPE/RMSNorm/SwiGLU) $\rightarrow$ 形状测试 $\rightarrow$ **改坏实验**: 去掉因果 mask、把 RoPE 换成可学习位置嵌入、去掉残差, 各跑一次看现象
- **判分证据**: tests/test_model.py、test_nn_utils.py 全绿 + 三组“改坏”实验的 loss 曲线与现象记录 (进证据/)
- **常见误解**: 以为 attention 是“可解释的检索”; 以为参数越多越好; 忽略 pre-norm 对稳定性的决定性作用

M4 • 训练: 数据、优化、系统

- **核心问题**: 为什么同样的模型有人训得动有人训崩? Scaling law 怎么指导算力分配? 多卡为什么不是线性加速?
- **关键概念**: 预训练目标与数据配比、去重与过滤、AdamW (解耦权重衰减)、Muon、LR schedule (warmup+cosine)、梯度裁剪、混合精度 (bf16/fp16/fp32 与 GradScaler)、梯度累积、DDP/FSDP/TP/PP、通信开销与扩展效率、Chinchilla 与 compute-optimal、--target_flops
- **对标**: CS336 A2/A3 + lecture 05-11; nanochat optim.py/base_train.py/runs/scaling_laws.sh; HF Ultra-Scale Playbook (分布式, 待核); 李沐 d2l ch11-ch12
- **亲手实验**: 手写 AdamW 与 LR 调度 $\rightarrow$ 单卡固定算力下的 depth sweep (d4/d6/d8/d10) $\rightarrow$ 拟合幂律外推 d12 $\rightarrow$ **用真实 d12 训练验证预测 (打脸链路)** $\rightarrow$ 双卡 DDP 测扩展效率
- **判分证据**: tests/test_optimizer.py 全绿; sweep 原始数据 + 拟合图 + 外推预测存档 (预测先写死再跑); 扩展效率曲线 + 通信占比
- **常见误解**: 以为“loss 不降就是数据问题”; 把显存不足当成硬件不够 (其实是批大小/精度/累积没调); 把 DDP 当免费加速

M5 • 推理与部署

- **核心问题**: 为什么生成是逐个 token 的? KV cache 省了什么、代价是什么? 量化会掉多少点? 吞吐与延迟怎么权衡?

- **关键概念**: 自回归解码、KV cache、批处理与 continuous batching、PagedAttention、采样策略 (temperature/top-p/top-k)、量化 (int8/int4/GPTQ/AWQ)、投机解码、显存占用分解、token 成本
- **对标**: nanochat engine.py (KV cache 实现) + infer_bench.py; CS336 lecture 10 (Inference); rasbt ch2 (生成与 KV caching)
- **亲手实验**: 从零写 KV cache 的自回归生成 → 对比有无 cache 的耗时/显存 → 手写一个贪心 vs top-p 采样对照 → 测不同 batch size 的吞吐
- **判分证据**: tests/test_engine.py 全绿 + 延迟/吞吐/显存三张图 + “cache 换来多少倍加速”的实测数字
- **常见误解**: 以为推理慢是因为“模型太大”(常是没批处理/没 cache); 以为量化无损

M6 · 后训练: 从“会说话”到“会做事”

- **核心问题**: SFT、DPO、RLVR 各自在补什么能力? 为什么 RLVR 能提升数学/代码却可能伤害通用能力? reward hacking 长什么样?
- **关键概念**: 指令微调与 chat template、损失掩码 (只算 answer 部分)、奖励模型、PPO、DPO 的隐式奖励、GRPO 的组内归一化优势、可验证奖励 (verifiable reward)、拒绝采样、蒸馏、合成数据、KL 正则与过度优化
- **对标**: CS336 A5 (含 test_grpo.py、GSPO/MaxRL); rasbt 《Reasoning from Scratch》ch3–ch8 (先建评测→推理时扩展→self-refinement→GRPO→改进 GRPO→蒸馏); Nathan Lambert 《RLHF Book》(免费在线, 17 章); HF LLM Course ch11–12; minimind train_dpo/train_grpo.py; nano-aha-moment (单文件单卡)
- **亲手实验**: 同一基座上做 SFT / DPO / GRPO 三变体 → 同一评测口径下对照 → 手写 GRPO 损失并逐项注释 (advantage 从哪来、为什么要除标准差) → 制造一次 reward hacking 并记录现象
- **判分证据**: tests/test_grpo.py 全绿 + 三方对照表 (同基座/同评测/同种子策略) + reward 曲线与失败样本分析
- **常见误解**: 以为 DPO 不用 reward model 就“更简单”; 以为 RL 一定比 SFT 强; 把训练 reward 上升当成能力提升 (不做 held-out 评测就是自欺)

M7 · 评测: 把“感觉变好了”变成“确实变好了”

- **核心问题**: benchmark 分数可信吗? 污染怎么判断? 为什么必须固定口径? 评测方差有多大?
- **关键概念**: bpb / perplexity / CORE / 下游基准 (GSM8K/MATH-500/MMLU/HumanEval)、pass@k、评测污染、few-shot 与 prompt 敏感性、方差与置信区间、LLM-as-judge 的偏置、可验证奖励 vs 学习型奖励
- **对标**: rasbt 《Reasoning from Scratch》ch3 (先建评测再上技术, 全书教学法基石); nanochat core_eval.py (单文件 CORE); CS336 lecture 12; Berkeley Agentic AI MOOC “Agent Evaluation”; lm-evaluation-harness
- **亲手实验**: 写一个“数学题判定器”(解析 + 数值比对 + 容错) → 测它在 200 道题上的准确率与假阳性 → 给自己的两个模型跑同一套评测并给误差棒
- **判分证据**: 判定器的单测 + 假阳性/假阴性样例分析 + 带误差棒的对照表 (不许只报均值)
- **常见误解**: 只看一个 benchmark 下结论; 忽略 prompt 格式带来的巨大差异; 把测试集当验证集反复调

M8 · 智能体：工具、记忆、harness、长程任务

- **核心问题**：LLM 从“回答问题”到“完成任务”缺什么？harness 为什么比模型本身更能决定成败？self-judge 什么时候有效、什么时候是自欺？
- **关键概念**：工具调用/函数调用、ReAct 循环、context engineering (Karpathy/李宏毅都专讲)、记忆与压缩、harness 设计（沙箱、权限、反馈信号）、长程任务的误差累积、多智能体、self-judge / self-refine 的边界条件、可验证环境
- **对标**：李宏毅 GenAI-ML 《Context Engineering》专讲；Berkeley LLM Agents MOOC + Agentic AI MOOC（含 “Post-Training Verifiable Agents”）；HF Agents Course（三大框架 + 排行榜）；rasbt《Reasoning from Scratch》ch5 (self-refinement)；PrimeIntellect-ai/verifiers、terminal-bench、SWE-bench
- **亲手实验**：手写最小 ReAct 循环（不用框架）→ 给它 2 个工具（计算器 + 代码执行）→ 记录失败轨迹分类 → 加 self-judge 再跑，做消融
- **判分证据**：轨迹日志（成功/失败分类）+ self-judge 有无的成功率差 + “什么时候 self-judge 有害”的案例
- **常见误解**：以为 Agent 的瓶颈在模型智力（常在你的工具接口与反馈信号）；以为 self-judge 免费提升（它会放大原有错误）

M9 · 前沿与安全（持续跟进区，不是一次性学完）

- **核心问题**：MoE/长上下文/多模态改变了什么？对齐与安全是“加一层”还是“改训练目标”？持续学习与“AI 训 AI”可行到什么程度？
- **关键概念**：MoE 稀疏激活、长上下文外推与记忆、多模态对齐、可解释性、红队与越狱、幻觉的成因、评测式安全保障、合成数据闭环、self-play、灾难性遗忘、模型合并
- **对标**：CS336 lecture 04 (MoE) + 论文精读清单；Berkeley Agentic AI MOOC（安全与红队）；opendilab/awesome-RLVR；LeapLabTHU/limit-of-RLVR
- **亲手实验（选题制）**：从“合成数据自训练一轮” / “拒绝采样提升数学” / “self-play 生成题目”三者中选一个，做最小闭环 + 反证
- **判分证据**：小实验 + 明确写出“这个方向的 no-go 是什么”（负结果不许擦除）
- **常见误解**：把“前沿”当成必须立刻掌握（正确姿势是保持索引能力 + 每季度更新一次本模块）

模块 ↔ 课题 ↔ 周次 对照

模块	主课题	周次	讲（对标）	做（手写区）	证（判分/证据）
M0	前置/贯穿	W1	d2l ch2-5	micrograd + MLP	数值梯度对照
M1	课题 02	W2	d2l ch8-10	bigram → RNN → Transformer	bpb 对照表
M2	课题 01	W1	CS336 A1 + HF ch2	手写 BPE	pytest 全绿 + 压缩率
M3	课题 02	W2-W3	CS336 lec03-04 + rasbt ch3-4	手写 Transformer	pytest + 改坏实验
M4	课题 03/04/06	W3-W9	CS336 A2/A3 + nanochat	AdamW + sweep + DDP	拟合图 + 扩展效率

模块	主课题	周次	讲（对标）	做（手写区）	证（判分/证据）
M5	课题 04/05	W5-W8	nanochat engine + CS336 lec10	KV cache + Triton kernel	三图 + 实测数字
M6	课题 08	W10-W12	理由书 ch3-8 + RLHF Book	SFT/DPO/GRPO 对照	test_grpo 全绿 + 对照表
M7	课题 07/08	W9-W12	Reasoning ch3 + CS336 lec12	判定器 + 评测	假阳性分析 + 误差棒
M8	课题 09/10	W13-W14	李宏毅 Context Eng. + Agents MOOC	最小 ReAct + self-judge	轨迹分类 + 消融
M9	课题 11（选题）	W15-W16	论文精读 + MOOC	选题小实验	no-go 记录

与课程（CORE100299）六板块的映射

课程板块	对应本图模块	课程大论文可用素材
绪论	M1	语言模型史一页
搜索求解	M8（Agent 的搜索/规划）	ReAct 与树搜索对照
知识表示	M8（记忆/工具）	context engineering 案例
机器学习（监督/深度/强化/无监督）	M0/M3/M4/M6	预训练+后训练全链路
智能体	M8	最小 ReAct + self-judge 消融
大语言模型	M2-M7	0.1B 端到端 + scaling law

#	资源	类型	语言	覆盖模块	核实	怎么用
7	李沐《动手学深度学习》(zh.d2l.ai, B站236集)	书 + 视频（免费）	中	M0/ M1/ M3/ M5	✓	“不是通过幻灯片讲解，而是通过解读代码、动手调参跑实验来学习”——中文知行合一的原典
8	rasbt《Build a Large Language Model (From Scratch)》+ rasbt/LLMs-from-scratch（105,140★, 2026-09-17 仍在更新）	书 + notebook	英	M2– M4, M6	✓	代码级最细；附录含 GQA/MLA/SWA/MoE/KV cache/FLOPs 分析/LoRA, 正好是 M3/M5 的补充读物
9	HF LLM Course (13 章, ch11–12 含 LoRA/SFT/GRPO)	课程（免费）	英	M2/ M6	✓	由 transformers/TRL 库作者写, ch8「调试训练流水线」是别处没有的
10	HF Agents Course (4 units + 最终项目排行榜 + function-calling 微调 bonus)	课程（免费, 有证书）	英	M8	✓	三个框架并学（smolagents/LlamaIndex/LangGraph），最后提交到公开排行榜——自带客观验收
11	Berkeley LLM Agents MOOC / Agentic AI MOOC (rdi.berkeley.edu、agenticai-learning.org)	公开课录像 + slides	英	M8/ M9	✓	有一讲就叫“Post-Training Verifiable Agents”、一讲“Agent Evaluation”——正是唐杰作业⑤的前沿坐标
12	Karpathy《Neural Networks: Zero to Hero》(micrograd/makemore/nanoGPT)	视频	英	M0/ M1/ M3	✓ 系列存在性已核	直觉地基；micrograd 是 M0 手写实验的模板（但你要自己写）
13	3Blue1Brown 神经网络系列	动画视频	英（有中字）	M0/ M3	⊖	视觉直觉；用来预热 attention/反向传播
14	HF Ultra-Scale Playbook	长文（免费）	英	M4	⊖	分布式训练（DDP/FSDP/TP/PP）的工程全景，W9 前核实
15	《大规模语言模型：从理论到实践》(复旦 张奇等)	书	中	M2– M6	⊖	中文教材备选；与 minimind 配套读

三、评测与环境类（M7/M8 的判分件）

#	资源	覆盖	核实	用途
16	harbor-framework/terminal-bench（+ 冻结版 terminal-bench-1）	M8	✓	长程 Agent 的连续基准（v2 新仓, v1 已冻结改名）
17	SWE-bench/SWE-bench、bench/SWE-smith	SWE-M7/ M8	✓	代码任务评测 / 合成可验证环境
18	GSM8K / MATH-500	M7	✓（在 rasbt ch3 用作 verifier 对象）	数学可验证奖励的标准件
19	PrimeIntellect-ai/verifiers (4,626★, MIT)	M7/ M8	✓ commit 已锁	RL 环境 + 评测的抽象层, 做课题 09 的参照

#	资源	覆盖	核实	用途
20	EleutherAI/lm-evaluation-harness、UKGovernmentBEIS/inspect_ai	M7	✓	学术评测标准件 / harness 设计参考
21	walkinglabs/hands-on-modern-rl (4,395★, 中文)	M6	✓	RL→RLVR→Agentic RL 中文教材, 实验标注 CPU/xGPU 且挂免费在线 Notebook
22	McGill-NLP/nano-aha-moment	M6	✓	单文件单卡从零 GRPO (阅读用, 不宜作依赖)

四、明确标注的坑（用前必读）

资源	坑
CS336 A3	需要 A3_API_KEY=8 位学号, 非斯坦福在校生拿不到 → 本学科用自建 sweep 替代 (课题 03 已按此设计)
CS336 A2	flash-attn 安装特殊: <code>uv sync --no-install-package flash-attn</code> 再 <code>uv sync</code>
CS336 A5	2026 版换成 Olmo-2-1B @ GSM8K, 且用 Modal 云函数 → 显存要求最高, 放在 W12 且用双 T4
Kaggle P100	若分配到 P100 (SM60), Triton 不可用 (需 CC≥7.0) → 课题 05 必须重开会话直到拿到 T4
karpathy/nanoGPT、nanogpt、llm.c	已被 nanochat 取代或停滞, 只作历史读物
karpathy/LLM101n	已归档 (archived), 且 Eureka Labs 至今未完整发布 → 只当索引, 别当路线
HuggingFace FineWeb	我用 <code>gh api</code> 查 HuggingFaceFW/fineweb、huggingface/fineweb 均 404 → 不要写这两个路径, 语料走 TinyStories / stanford-cs336/owt-sample / dolma
terminal-bench	已改名分仓 (v1 → terminal-bench-1, v2 → 新仓), 旧链接会 404

五、三师制使用方法（每个知识点的标准动作）

- ① 先读对标材料的那一节（不超过 25 分钟，带着“我要回答哪 3 个问题”去读）
- ② 合上材料，用自己的话讲给我听（我按 M0–M9 的“核心问题”追问，专挑你含糊的地方）
- ③ 我讲你的卡点（只讲你卡住的那一层，不重讲整章）
- ④ 上手写（手写区任务，最小可运行）
- ⑤ 跑判分 (pytest / 自建判定器) → 出证据
- ⑥ 改坏它 → 预测 → 验证 → 修好 (第 3 条硬标准)

分工原则：对标名师负责“标准讲法”，我负责“你的卡点 + 判分 + 记忆归档”，任何一方都不替代你动手（见《造轮子边界.md》）。

知行合一规程（本学科教学法核心）

立规 2026-09-17。缘起：用户指出“实操性够强，但知识讲解偏少”，并明确要求把知行合一贯穿全流程。本规程定义：什么算“学过”，什么算“学会”，以及每一层怎么被强制咬合。

一句话：本学科没有“听懂了”这个状态，只有在代码里被验证过的“懂了”。

一、把“知行合一”变成可执行的四件咬合

咬合件	名称	谁的活	不通过的后果
讲	导师讲 + 对标名师	我（≤20 分钟一块，≤150 字一个概念块）	讲完必须立刻出“代码级提问”
做	手写区最小可运行	你	没跑起来 = 这一课不算上
证	判分器 / 实验证据	我搭台，你跑	无证据的结论一律不采纳
反	改坏实验 + 打脸记录	你预测，你验证	只会正确写法的人，遇到 bug 一定卡死

四件缺一 → 本知识点标 ❶ 未通过，不进 PROGRESS 的 ✓。

二、知识卡规范（每个知识点一张，落 知识卡 / 或课题目录）

知识卡 · <知识点名>（模块 M?）

- 一句话定义：
- 直觉（给别人讲会用的话）：
- 公式/伪代码骨架（手写，不许贴代码）：
- 对标出处：<材料名 + 章节>
- 上手写实验：<文件路径 + 判分命令>
- 判分证据：<pytest 名 / 图 / 数字>
- 改坏实验：改哪里 → 你预测的现象 → 实测现象
- 常见误解：
- 记忆锚（隐喻+口诀+符号位置）：
- 间隔重复卡号：

知识卡由你口述、我整理（你说我写），但“直觉”“改坏实验的预测”两栏必须是你自己的话。若某栏你说不出来 → 这节课没上完。

三、讲课节拍（每周两个时段，各 1.5–3h）

时段 A（理论）：讲（≤20min 一块）→ 你复述 → 我挑含糊处追问 → 你写知识卡 → 出代码级提问

时段 B（实验）：读对标材料对应节 → 你写手写区 → 跑判分 → 记录证据 → 改坏实验 → 归档

硬纪律：

- 一次讲透，不挤牙膏（沿用你已定的节奏：你多讲、我多写；批量 3–5 个关键问题）。
- 每个概念块 ≤150 字，讲完必须配一个“立刻能跑的最小实验”。
- 讲课时不许给手写区代码（见《造轮子边界.md》）；给的是“该去哪里找、该测什么”。

四、代码级提问（“知”与“行”的咬合点，本学科独创）

每讲完一块，我出 3-5 道代码级提问，形式固定为“改坏它 / 预测现象 / 给出诊断”：

类型	例子
改坏预测	“把 attention 的因果 mask 注释掉，loss 会怎样？训练 100 步内你会看到什么现象？”
放大器	“把 RoPE 换成可学习位置嵌入，在训练/外推两种场景下各自会怎么退化？”
诊断	“你观察到 loss 在第 400 步突然变 NaN，请按顺序列出你会检查的 5 个量。”
代价	“把词表从 32k 扩到 128k，参数量、序列长度、每 token 成本各变多少？（估数量级）”
反事实	“如果 DDP 改成 FSDP，同样的 batch，显存和速度分别会怎么动？为什么？”

判分标准：预测错不算失败（预测错 + 跑实验打脸 = 最高质量的学习），但“没有预测就跑”算失败，因为那等于放弃了知行之间的连接。

五、“真的懂了”的三条硬标准（与《知识地图》一致，此处为验收动作）

1. **能讲清** → 你对着外行话讲一遍，我专门装不懂追问三层（Why → Why not → What if）。
2. **能指出代码位置** → 你在我画出的手写/或上游代码里，**不搜索**直接指出对应函数/行号区间。
3. **能改坏并修好** → 我在你没看过的地方埋一个 bug（或你自己改坏），你在**不看不该看的代码**的前提下，用实验定位并修好；修好过程必须留日志进证据/。

三条全过才写 ✓；只过第 1 条 = 会考试，不会干活。

六、反讲机制（费曼 + 红队）

- 每模块收口：**你当老师，我当刺头学生**，我只问“为什么”，不纠正、只追问，直到你卡住或讲穿。
- 卡住的地方就是下一个知识点的入口，直接写进 HANDOFF.md 的“下一步”。
- 结项前：你来当评审，对着自己的报告做 3 条自我攻击（“这个结论最可能错在哪？”），与导师审稿意见并列存档 → 这就是唐杰作业⑦“问题/动机/方法/结果一项都不能糊”的本地化实现。

七、与其它章程的关系

- 谁写代码 → 《造轮子边界.md》（R1-R10 手写 / B1-B8 代写 / C 协作）
- 讲什么、讲多深 → 《知识地图-大模型全景.md》（M0-M9 四件咬合件）
- 用哪份材料讲 → 《对标教学资源清单.md》（三师制）
- 怎么算过 → 《判分与验收标准.md》（三层判分 + 每课题通过证据）
- 什么时候讲 → 《实训路线图.md》“知识讲解锚点”列

实训路线图 · 16 周 × 11 课题 × 唐杰 7 条 × CS336 映射

2026-09-17 并入人工智能基础与应用/（实训轨）：本学科三轨合一——课程轨（CORE100299 课堂事务，见 ../课程轨/课程进度.md）、实训轨（本文件 + 11 个课题）、知识轨（知识地图-大模型全景.md M0-M9）。配套：《对标教学资源清单.md》（跟谁学）、《知行合一规程.md》（什么叫学过）、《造轮子边界.md》（谁写代码）、《算力与成本预算.md》（花多少）、《判分与验收标准.md》（怎么算过）。
立图 2026-09-17。基准：第 1 教学周 = 2026-09-14（与校历、与唐杰清华开课同周）。节奏：16 周跟课（每周 5-6h，约 2 个学习时段）。预算：零现金，只用免费算力（见《算力与成本预算.md》）。

一、目标（一句话）

用零现金预算，把唐杰 2026 秋季作业清单一条不落地走完，每一环都留下可复现证据、可执行判分与一页报告；最终产出一份 NeurIPS 格式的英文技术报告 + 一个可现场 demo 的作品。

二、唐杰 7 条 ↔ 本学科课题 ↔ 判分来源（主映射表）

唐杰作业条目	对应课题	官方规格 / 判分器	算力档
① 从零写 Tokenizer + Transformer, 端到端训 0.1B	课题 01、02、04	CS336 A1 (tests/)	T0 → T2
② 手写 Triton attention kernel, 测多卡训练/推理增益	课题 05、06	CS336 A2	T1 / T2
③ raw dump 洗语料, 拟合 scaling law 再外推	课题 07、03	CS336 A4 + A3 (A3 需自建替代)	T0 / T1
④ 同一基座做 SFT / DPO / RLVR 对照	课题 08	CS336 A5 (含 tests/test_grpo.py)	T2
⑤ 可验证环境 + harness, 训长程 Agent, 鼓励 self-judge loop	课题 09、10	无官方判分器（前沿区）→ verifiers / terminal-bench 思路	T2
⑥ 2-3 人组队, 英文 NeurIPS 格式, W16 现场 demo	课题 11	本仓库报告协议 + LaTeX 模板	—
⑦ 作业 40% / 大项目 60%; 问题/动机/方法/结果一项都不能糊	全学科判分口径	《判分与验收标准.md》	—

三、16 周课表（周次 / 课题 / 交付物 / GPU 需求）

周	日期	课题	本周交付物（硬指标）	GPU
W1	09-14→09-20	课题 01 BPE 分词器	手 写 BPE + test_tokenizer.py/ test_train_bpe.py 全绿 + 压缩率表	0
W2	09-21→09-27	课题 02 Transformer 骨架	attention/MLP/RoPE/RMSNorm 手写 + 形状与数值测试过	0
W3	09-28→10-04	课题 02 收口（国庆周，减半）	训练循环 + AdamW + 序列化测试过；loss 稳定下降曲线	0
W4	10-05→10-11	课题 03 Scaling Law 预演	5 点小规模 sweep + 拟合 + 外推预测 + 打 脸记录	0 (CPU)
W5	10-12→10-18	课题 04 0.1B 端到端（上）	数据管线 + 双 T4 冒烟 + 首轮训练启动（断 点续训可用）	6-9h (2×T4)

周	日期	课题	本周交付物（硬指标）	GPU
W6	10-19→10-25	课题 04 收口	bpb/CORE 指标 + 采样样本 + 实测成本账单 vs 估算	6-12h (2×T4)
W7	10-26→11-01	课题 05 Triton kernel (上)	naive 实现 → 分块实现, 正确性对齐	4h (1×T4)
W8	11-02→11-08	课题 05 收口 + 课题 06 开题	speedup / 显存 / profiling 三张图 (含硬件口径声明)	6h (2×T4)
W9	11-09→11-15	课题 06 多卡与分布式	DDP vs FSDP 对照 + 扩展效率曲线 + 通信占比分析	8h (2×T4)
W10	11-16→11-22	课题 07 数据清洗与去重	过滤/去重规则 + 有无清洗的对照组 loss 对比	6h (1×T4)
W11	11-23→11-29	课题 08 SFT	SFT 基线 + 与基座对照	8h (2×T4)
W12	11-30→12-06	课题 08 DPO + RLVR	SFT / DPO / GRPO 三方对照表 (同基座、同评测口径)	10h (2×T4)
W13	12-07→12-13	课题 09 可验证环境 + harness	1 个自建可验证环境 + rollout 成功率 + 判定逻辑单测	6h (2×T4)
W14	12-14→12-20	课题 10 长程 Agent + self-judge	多步任务串联 + self-judge 有无的消融	8h (2×T4)
W15	12-21→12-27	蓄水周	补跑欠账 + 英文报告初稿	4h (弹性)
W16	12-28→01-03	课题 11 结项	NeurIPS 格式英文报告 + 现场 demo + 总复盘	4h (弹性)

GPU 总需求 ≈ 97 小时, 免费额度可用 **≈480 小时** (Kaggle 30h/周 × 16 周) → 安全系数约 5× (见预算章程)。

附：知识讲解锚点（每周讲哪个模块——补上“讲”这条腿，2026-09-17 新增）

每个知识点的“四件咬合”（讲/做/证/反）见《知行合一规程.md》；模块内容见《知识地图-大模型全景.md》；对标材料见《对标教学资源清单.md》。

周	讲解模块	本周要讲透的核心问题（≤5 个）	配套材料
W1	M0 + M2	计算图与链式法则；熵/交叉熵/困惑度；BPE 为什么字节起步	d2l ch2-5；CS336 A1 handout
W2	M1 + M3 (上)	从 n-gram 到 attention 的动机；attention 在算什么；为什么多头	d2l ch8-10；CS336 lec01/03
W3	M3 (下)	RoPE vs 可学习位置；RMSNorm/pre-norm 为何关键；残差防什么	CS336 lec03-04；rasbt ch3-4
W4	M4 (上)	优化器与 LR 调度；混合精度；scaling law 如何指导算力分配	CS336 lec09/11；nanochat optim.py
W5	M4 (下) + M5 (上)	数据配比与去重；吞吐/显存/FLOPs 记账	CS336 lec05/13-14；nanochat base_train.py
W6	M5 (下)	KV cache 省了什么；采样策略；bpb/CORE 指标口径	nanochat engine.py/core_eval.py；CS336 lec10/12
W7	M5 (续·kernel)	GPU 内存层级；Triton 编程模型；flash attention 的分块与 online softmax	CS336 lec05-06；triton 官方教程
W8	M4 (续·并行)	DDP/FSDP/TP/PP 的取舍；为什么不是线性加速	CS336 lec07-08；nanochat optim.py

周	讲解模块	本周要讲透的核心问题（≤5 个）	配套材料
W9	M7（上）	评测口径、污染、方差、误差棒；“变好了”如何被证明	rasbt ch3; CS336 lec12
W10	M7（下）+ M6（上）	过滤/去重的判定规则；指令微调与损失掩码	CS336 A4; 《RLHF Book》ch4
W11	M6（中）	偏好数据、奖励模型、DPO 的隐式奖励	《RLHF Book》ch5/8/11; CS336 A5
W12	M6（下）	GRPO 的组内优势、KL 正则、reward hacking、过度优化	rasbt ch6-7; 《RLHF Book》ch6/7/14
W13	M8（上）	工具调用/ReAct/context engineering; harness 为什么决定成败	李宏毅《Context Engineering》; Agents MOOC
W14	M8（下）	长程任务的误差累积; self-judge 何时有效、何时有害	rasbt ch5; Berkeley Agentic AI MOOC
W15-16	M9（选题）	前沿采样: MoE/长上下文/合成数据闭环; no-go 的写法	论文精读清单; awesome-RLVR

四、11 课题总表（手写要点 / 判分来源 / 通过证据）

#	课题	手写区核心（不可代写）	判分来源	通过证据
01	BPE 分词器	merge 循环、特殊 token、bytes 回退、流式编码	CS336 A1 tests/test_tokenizer.py、test_train_bpe.py	pytest 全绿 + 压缩率表
02	Transformer 骨架	attention、RoPE、RMSNorm、SwiGLU、残差、初始化	CS336 A1 test_model.py、test_nn_utils.py	pytest 全绿 + 参数量核对
02b	训练循环与优化器	AdamW/Muon 更新式、LR schedule、梯度裁剪、精度管理	CS336 A1 test_optimizer.py、test_serialization.py	pytest 全绿 + loss 曲线
03	Scaling Law	数据采集脚本口径、拟合、外推、打脸记录	自建判分：预测 vs 实测偏差	5 点 sweep 数据 + 拟合图 + 外推预测存档
04	0.1B 端到端	（复用 01/02 产物）预训练编排、断点续训判断	自建判分：loss/bpb 达标 + 采样可读	bpb/CORE + 采样样本 + 成本账单
05	Triton attention kernel	kernel 本体（分块、online softmax）	CS336 A2 + 正确性对齐参考实现	三图（speedup/显存/profile）+ 数值误差记录
06	多卡与分布式	并行策略选择、通信原语、扩展效率分析	CS336 A2 分布式部分 + 自建 benchmark	扩展效率曲线 + 通信占比
07	数据清洗与去重	过滤/去重规则的判定逻辑	CS336 A4 + 自建消融	规则清单 + 对照组 loss 差
08	SFT / DPO / RLVR	损失函数与优势估计（A/group norm/重要性比）	CS336 A5 tests/test_grpo.py + 自建对照	三方对照表（同口径）
09	可验证环境	判定逻辑（verifier/reward）	自建单测 + rollout 成功率	判定逻辑测试 + 成功率 + 失败样本分析
10	长程 Agent	self-judge 协议设计	自建消融	有无 self-judge 的成功率差 + 案例
11	结项	报告骨架与结论判断	报告协议 + 导师审稿	英文报告 + demo

五、规模三档（零预算版）

档	硬件	模型规模	数据	用途
T0	沙箱 CPU (2 vCPU/3GB) 或 Colab CPU	1–10M	TinyStories 子集 / owt-sample 采样	课题 01–03 的全部正确性与判分
T1	Kaggle 1×T4 16GB	0.03–0.1B	TinyStories 全量 + 子集	课题 05/07 的单卡实验
T2	Kaggle 2×T4 (32GB 合计)	0.1B	同上 + 合成任务数据	课题 04/06/08/09/10

规模下探是主动取舍，不是妥协：把“0.1B”从口号变成每次会话都能真跑的实验，代价是学不到万卡级工程；收益是 16 周内全链路闭环。前沿规模直觉另走 论文精读/ 补齐。

六、跨学科搭桥（讲课时主动用）

- RL 的目标函数 → probability/ 的期望、方差、条件期望 (REINFORCE = 用样本估期望)
- 语言模型的困惑度/交叉熵 → probability/ 的信息量直觉
- scaling law 的幂律拟合 → probability/ 的最小二乘与回归
- 结项报告 → 论文精读/ 的论文写作规范；demo → 人工智能基础与应用/ 的大论文素材

造轮子边界 · 谁写代码（本学科最高契约）

立约 2026-09-17，用户拍板口径：“分层双轨”。本学科同时要求两件互相冲突的事：① 唐杰作业要求“不会动手、只会调 API 的先下去”；② 你的时间是稀缺资源，16 周内跑完全链路。**解决办法不是折中，而是划死边界**：学习目标必须你手写；非学习目标必须我代写。边界之下，任何一方越界都算事故。

一、三条铁律

- 1. **手写区（课题 NN/手写/）：Agent 禁止写入任何文件。**不得给代码、不得给伪代码、不得把第三方实现（含 nanochat / CS336 参考实现 / GitHub 上的答案库）当答案指给你。允许的动作只有五类：解释报错、解释概念、审查你已写的 diff、建议 sanity check / toy example / profiler 检查、指路官方文档与讲义。
- 2. **脚手架区（课题 NN/脚手架/、环境/）：Agent 全权负责且必须主动完成，不等你催。**这部分写一遍就够，重复劳动零学习收益。
- 3. **越界的正确姿势：**你说“给我看答案”→ 走提示阶梯（五级）→ 到第五级给答案后，**必须挂近迁移重考卡片**（隔 1-3 天、无提示、换场景再考）。给了答案不挂卡片 = 违规。

二、手写清单（学习目标，共 10 项）

编号	手写内容	为什么必须亲手写
R1	BPE 训练与编码算法（merge 循环、特殊 token、bytes 回退、流式编码）	分词决定了模型看到的世界；面试/科研高频
R2	Transformer 全部组件（attention、RoPE、RMSNorm、SwiGLU、残差、初始化）	梯度与形状直觉只从亲手写里长出来
R3	训练循环与优化器（AdamW/Muon 更新式、LR schedule、梯度裁剪、精度管理）	训练不收敛时，只有写过的人会 debug
R4	反向传播验证（gradient check、数值梯度对照）	这是“我懂了”与“我能跑”的分界线
R5	Triton attention kernel 本体 （分块、online softmax）	唐杰作业②的核心；性能直觉的唯一来源
R6	Scaling law 拟合与外推（数据采集口径、拟合、预测、打脸记录）	这是“科研判断力”的训练区，不能外包
R7	SFT / DPO / RLVR 的损失函数与优势估计（含 group normalization、重要性比）	后训练是 2026 的重心；抄公式等于没学
R8	数据过滤/去重的判定规则（启发式规则设计）	数据决定上限；规则设计的品味靠自己做
R9	可验证环境的判定逻辑（verifier / reward function）	唐杰作业⑤的技术核心
R10	self-judge loop 的协议设计（何时判、判什么、如何防退化）	唐杰课明确鼓励的方向；前沿无标准答案

三、代写清单（脚手架，Agent 负责，共 8 项）

编号	代写内容	交付形态
B1	环境与依赖（uv / CUDA / 镜像 / 引导脚本）	环境/bootstrap*.sh + uv.lock
B2	数据下载/解压/分片/上传（含 Kaggle Dataset 打包）	环境/数据/脚本 + 清单
B3	日志、曲线绘图、实验编排、断点续训封装	课题 NN/脚手架/脚本
B4	判分脚本（拉取 CS336 官方 tests/ 并接入适配器）	环境/judge/ + judge.sh

编号	代写内容	交付形态
B5	证据带自动归档（run.yaml / 日志 / 曲线 / 产物哈希）	课题 NN/证据/ 由脚本生成
B6	报告模板与排版（Markdown + LaTeX/NeurIPS 模板）	环境/报告模板/
B7	配额与超时治理（会话预检、超时自停、配额账本）	环境/预检.md + 脚本
B8	checkpoint 上传/下载与版本管理	课题NN/脚手架/ckpt_*.sh

四、协作区（一起定，你拍板）

编号	事项	分工
C1	架构与底座选型、上游引入范围	Agent 出方案与证据 → 你拍板
C2	超参搜索空间、指标口径（bpb / CORE / 通过率）	Agent 出方案 → 你定口径
C3	实验优先级与是否值得重跑	Agent 给成本估算 → 你决定
C4	可验证环境的题目设计	你出设计意图 → Agent 评估可行性与成本

五、违规示例（判例，会长大）

- **E-01（预防性记录）**：Agent 往 手写/ 里写入实现文件 —— 即使内容正确也算违规，必须改为“审查意见 + 测试建议”落到 判卷记录.md。
- **E-02（预防性记录）**：Agent 说“你可以参考 nanochat 的 flash_attention.py 第 30 行” —— 违规。正确做法：说明该文件在整体架构中的位置与接口契约，指出官方 Triton 教程与 CS336 A2 handout 的对应章节。
- **E-03（预防性记录）**：Agent 帮你把实验卡写好之后直接开始烧 GPU 时间 —— 违规。实验卡必须由你确认（哪怕只是回一句“跑”）。

六、边界自检（每次动手前问一遍）

1. 我现在要写的这个文件，属于 R1-R10 还是 B1-B8？
2. 如果是 R 类 → 我只能读、只能评论，不能写。
3. 如果是 B 类 → 我必须写，且要写完整（不留 TODO 给用户）。
4. 如果两边都不是 → 进协作区，先出方案再动手。

AI 使用红线（本学科版）

改编自 Stanford CS336 官方 AGENTS.md (stanford-cs336/assignment1-basics, 2026-09-17 读取原文), 按用户的“分层双轨”口径改写。CS336 原文的立场是“Teaching Assistant, Not Solution Generator”, 本学科的立场是“学习目标区=TA, 基建区=工程师”。

一、最高原则

学习目标 (R1-R10) 必须由用户亲手完成；非学习目标 (B1-B8) 必须由 Agent 完成。边界见《造轮子边界.md》。任何一端的越界都算事故，写进 memory/ERRORS.md。

二、手写区：Agent 应当做 (SHOULD)

- 解释概念：只讲用户卡住的那一层，不重讲整章；用一个反例或一个玩具例子收尾。
- 指路：指出对应讲义章节 (CS336 / d2l / 李宏毅 / rasbt / RLHF Book 的具体哪一节) 与官方文档。
- 审查用户代码：提 3-5 条意见，指向“哪个不变量可能被破坏”“哪个边界情况没测”，不直接给修好的代码。
- 帮助 debug：问引导性问题（“你检查过 mask 施加在 softmax 之前还是之后吗？”），而不是给修复。
- 解释报错：Python / PyTorch / CUDA / Triton / 分布式栈的报错含义与排查顺序。
- 建议验证手段：形状断言、玩具输入、profiler 检查、消融、梯度检查（数值对照）。

三、手写区：Agent 禁止做 (SHOULD NOT, 照抄 CS336 并加严)

- ✗ 写任何 Python 或伪代码
- ✗ 给出问题的解法或思路解法
- ✗ 完成手写区里的 TODO
- ✗ 编辑用户仓库里的手写区文件
- ✗ 大段重构用户代码变成“成品答案”
- ✗ 把作业要求直接翻译成可运行代码
- ✗ 替用户实现核心组件 (tokenizer、transformer 块、优化器、训练循环、Triton kernel、分布式逻辑、scaling law 管线、数据过滤/去重、对齐/RL 方法)
- ✗ 把第三方实现当答案指路 (CS336 原文：课程材料自包含) ——本仓加严：包括 nanochat / minimind / GitHub 上的作业答案库。可以指出“该功能在架构中的位置与接口契约”，但不得指出“去抄哪几行”。

四、脚手架区：Agent 应当做（且必须做完）

- ✓ 环境/：依赖锁、引导脚本、会话预检、报告模板、数据打包
- ✓ 脚手架/：实验编排、日志、曲线绘图、断点续训、checkpoint 管理、判分脚本、证据归档
- ✓ 课程轨/：课程进度台账、大论文模板与格式整理
- ✓ memory/：HANDOFF / PROGRESS / ERRORS / LEARNINGS / MEMORY / drill-ledger 的维护

判据：脚手架代码里不允许留 TODO 给用户——那是把学习目标混进了基建。

五、例外与补偿（本学科特有，CS336 没有）

情形	允许	补偿动作
用户明说“卡死了，给我看答案”	走提示阶梯五级，第 5 级给答案	必须挂“近迁移重考”卡（隔 1-3 天、无提示、换场景复考）
用户连续 2 轮在同一处打转	由我提议“降档”：给一个更小的可跑通示例（不是我写他的目标代码）	记录到 ERRORS.md，下一课换讲法
环境/工具链阻塞（非学习目标）	直接由我修好，不占用用户时间	记入 证据/ 的环境变更

六、教学法提示（照抄 CS336 的方法论，本学科已内化）

用户求助时，按此顺序：①问澄清问题（试过什么/期望什么/发生了什么）→ ②引用讲义概念而非直接回答 → ③建议下一步而不是替他做 → ④审查代码并指出可能 bug/缺失检查的方向性提示 → ⑤解释“为什么”而不只是“怎么做” → ⑥优先用测试与不变量而非修复。

七、学术诚信与“知行合一”的关系（本学科的立场）

CS336: AI 只能用于低层次编程问题与高层次概念问题，不得直接解题；“我们强烈建议在完成作业时关掉 AI 自动补全（Cursor Tab / Copilot）——我们发现它让人更难深入内容。”

本学科采纳这条建议的一半：动手写 R1-R10 时，手写区文件的编辑器内 AI 自动补全也要关（或在脚手架/里明确标注禁止生成手写区文件）。理由：本学科的验收标准是“能改坏并修好”——如果代码是补全出来的，你就没有能力在它坏掉时诊断它。

八、越界举报（自检清单，每轮自问）

1. 我这一轮写的文件，属于 R 类还是 B 类？
2. 我给的是“问题”还是“答案”？（给答案前是否走了阶梯、是否挂了重考卡？）
3. 我有没有指路第三方实现？
4. 用户这一轮有没有“预测 → 验证”的环节？没有的话，我是不是又变成讲解器了？

算力与成本预算（零预算版）

立规 2026-09-17。用户拍板：**零现金预算**，但要求**全量对标唐杰 7 条**（含多卡与长程 Agent）。本文件证明这个组合可行，并定义它的代价与纪律。

一、免费资源池（2026-09-17 核实）

平台	免费额度	硬件	会话上限	持久存储	卡 要 求	角色
Kaggle	30 GPU·小时/周（每周重置）	2×T4（32GB 合计）或 1×P100	9–12h（支持后台运行、每 5 分钟自动存档）	20GB kaggle/working	/ 无需	主战场（唯一能做“多卡”的免费档）
Google Colab 免费	~15–30 GPU·h/周（未公布，随需求波动）	1×T4 16GB	12h, 可能提前断	Google Drive（易失）	无需	备用（短实验、CPU 实验）
沙箱 CPU（本仓）	无限	2 vCPU / 3GB RAM	—	仓库（注意体积分纪律）	无需	T0 全部课题的正确性与判分
Lightning AI 免费档	约 15 credits/月	T4/A10G 等	~3–4h 重启	50GB	无需	备用（额度与 GPU 型号未核实，用前先验）
SageMaker Studio Lab	4h/次、4h/24h	1×T4	4h	15GB	需 账号	极端备用
Modal	3 ⁰ / _月 免费额度（CS336 赞助商 B200 6.25/h ≈ 4.8h/月）	2×B200	按任务	卷	需 国 际 支 付 / 网 络	可选升级，非计划内

结论：16 周可用 ≈ 480 GPU·小时（Kaggle 30h × 16 周），本学科计划只用 ≈ 97 小时 → 安全系数约 5×。

二、三档规模（零预算下的能力边界）

档	硬件	模型规模	典型任务	单次时长
T0	沙箱 CPU / Colab CPU	1–10M	课题 01–03 全部、判分器、知识卡实验	分钟级
T1	Kaggle 1×T4 16GB	0.03–0.1B	课题 05 Triton bench、课题 07 数据消融	1–4h
T2	Kaggle 2×T4 32GB	0.1B	课题 04 端到端、课题 06 DDP、课题 08 RL、课题 09/10 Agent	6–12h

三、成本估算（标记为估算，必须在课题 04 用实测替换）

- 参考点（实测自 nanochat 公开文档）：**d12 ≈ GPT-1 量级 ≈ 0.1B**，在 8×H100 上约 **6 分钟**。
- T4 是 **SM75**，无 **bfl6**（nanochat 自动退化 fp32，需手动 fp16 + GradScaler 才能提速）；等效算力约为 H100 的 1/15–1/25。
- 估算：单 T4 约 12–20h，双 T4 DDP 约 **7–12h**（扩展效率按 60–80% 估）。
- 执行方式：拆成两个 9h 会话（Kaggle 会话上限 9–12h），中间用 /kaggle/working 的 checkpoint 续训。

- △ 这个数字不许照抄进报告：课题 04 结束后，用实测耗时替换，并在报告里写明“估算 vs 实测”的偏差。

四、每周配额分配（16 周）

周	课题	计划 GPU 用量	备注
W1-W4	课题 01-03	0 h	纯 CPU, 零消耗期(先把正确性做对)
W5	课题 04（上）	6-9 h	数据管线 + 双 T4 冒烟 + 首轮训练
W6	课题 04（下）	6-12 h	收口 + 采样 + 成本账单
W7	课题 05（上）	4 h	单 T4（必须是 T4，P100 不可用）
W8	课题 05（下）+ 课题 06 开题	6 h	Triton bench 三图
W9	课题 06	8 h	双 T4 DDP/FSDP 对照
W10	课题 07	6 h	单 T4 数据消融
W11	课题 08（SFT）	8 h	双 T4
W12	课题 08（DPO+RLVR）	10 h	双 T4，最贵的一周
W13	课题 09	6 h	双 T4 rollout
W14	课题 10	8 h	双 T4
W15-W16	蓄水/补跑/结项	4+4 h	弹性
合计		≈97 h / 480 h	余量用于重跑与失败实验

五、硬纪律（出过事故的领域）

1. 实验卡制度：上 GPU 前必须先写实验卡，经用户确认才开跑。模板：

```
### 实验卡 #NN（课题NN / 模块 M?）
- 假设（一句话，可被证伪）：
- 命令（完整可复制）：
- 预算上限：≤ ? GPU·小时（默认 3）
- 停止条件：loss 不降 / 超过预算 / 出现 NaN → 立即停
- 成功判据（跑之前先写死）：
- 预期现象（**先写预测**，跑完对照——这是知行合一的咬合点）：
- 产物归档：证据/run.yaml + 日志 + 曲线 + 哈希
```

1. 会话预检(每次开 Kaggle 先跑):nvidia-smi 看型号 → **T4 或 2×T4 才继续**;**P100 立即重开会话**(SM60, Triton 不可用，课题 05 会在上面白烧配额)。
2. 断点续训是强制项: Kaggle 会话随时可能断; 每 N 步写 checkpoint 到 /kaggle/working (20GB 持久)。
3. 超时自停: 脚本内自带 wall-clock 上限（默认 3h），到点保存退出，不靠人盯。
4. 配额账本: 跑完立刻在 memory/PROGRESS.md 记：日期 / 平台 / GPU 型号 / 实际耗时 / 累计消耗 / 本周余量。
5. 不做无判据的实验: 没有“成功判据”就不许开跑——这是防止“烧配额摸鱼”的唯一有效办法。

六、可选升级路径（仅在零预算实在卡住时启动，需用户拍板）

方案	价格（2026 公开比价，下单前以平台现价为准）	何时值得
国内云 4090 按量	¥1.45-2.0/小时（AutoDL / 恒源云 / 智星云等）	双 T4 跑不动 0.1B RL（W12）时；约 ¥30 可买 15h
Modal 免费额度	\$30/月（≈4.8h B200）	需要 bf16 + 快迭代时（需国际支付）
Colab Pro	订阅制	不建议（免费档足够本学科用量）

诚实声明：本学科的设计前提是“**免费额度足够**”，依据是 § 三 的估算与 § 四 的配额表。若课题 04 实测发现双 T4 需要 >20h，则触发“规模下探一级”（d12 → d10，或语料减半），**而不是**自动升级付费；升级必须由用户单独拍板。

判分与验收标准

立标 2026-09-17。继承仓库总纲 skills/科研式学习导师/SKILL.md § 5a 《命题与判卷纪律》，并按本学科“可执行判分”的特性做加法。与总纲冲突处，以本文件的更严条款为准。

一、三层判分（每一层都必须留下原始输出）

层	谁判	判据	归档
L1 自动判分	判分器（pytest / 自建 script）	通过 / 失败 / 覆盖率 / 指标数字	判卷记录.md 贴原始输出（不许转述）
L2 导师审稿	我	3-5 条：必考项 / 要记住 / 预告项（含“哪里可能在自欺”）	判卷记录.md
L3 费曼收口	你讲 + 我追问	三条硬标准（讲清 / 指代码 / 改坏修好）	判卷记录.md + 知识卡

L1 不通过 → 不进 L2/L3；L3 不通过 → 课题不标 ✓，写进 HANDOFF 的“下一句”。

二、通用红线（继承总纲 + 本学科加严）

- 结果对 ≠ 满分：本学科分数按“证据链”给，逻辑链完整才满分（总纲 § 5a 第 5 条）。
- 没有预测的实验不算实验：跑之前必须先写“预期现象”（见实验卡），否则 L1 直接判不合格。
- 无证据的结论不采纳：曲线要原始日志/repro 命令；截图要能追溯到 run.yaml。
- no-go 不擦除：失败的实验、被推翻的假设全部留在证据/，并在报告里如实呈现。
- 口径统一：跨模型比较必须同数据集、同 token 化、同评测脚本；指标优先用词表无关的 bpb。
- 不许自创指标：新增指标必须给出定义 + 为什么已有的不管用 + 与已有指标的相关性。
- 误差棒是必填项：任何“变好了”的结论都要给多次运行的范围（至少 2 个种子）。
- 不许把测试集当验证集反复调（M7 常见自欺）。
- 掌握的定义（总纲 § 5a 第 8 条原文）：当次对 + 隔 1-3 天无提示再对 + 能讲清为什么。
- 宣布“通过”必须附证据引用：哪份判分输出、哪个数字、哪些 leech 已清零。

三、每课题的“通过证据”清单（缺一不可）

#	课题	L1 自动判分	L2 审稿关注	L3 收口动作	额外证据
01	BPE 分词器	test_tokenizer.py + test_train_bpe.py 全绿	边界：空串、单字节、中文、emoji、往返	手写一段中文的编码过程并解释	压缩率表（中/英）+ 往返一致性
02	Transformer 骨架	test_model.py、test_nn_utils.py 全绿	形状/初始化/数值稳定性	白板上画出一向前向的数据流	三组改坏实验（去 mask / 换位置编码 / 去残差）曲线
03	Scaling Law	自建：拟合优度 + 外推误差	数据点是否同口径、是否外推过远	复述“为什么用幂律而不是线性”	5 点 sweep 原始数据 + 预测先写死再跑的记录
04	0.1B 端到端	自建：loss/bpb 达标 + 采样可读性	是否过拟合/是否记住了测试集	讲清“这个模型会什么、不会什么”	采样样本 + 实测成本 vs 估算偏差
05	Triton kernel	CS336 A2 正确性对齐（误差 < 容差）	数值稳定性、边界 block、dtype	讲清 online softmax 为什么必要	speedup/显存/profile 三图 + 硬件口径声明

#	课题	L1 自动判分	L2 审稿关注	L3 收口动作	额外证据
06	多卡与分布式	扩展效率数据 + 梯度一致性检查	通信占比是否解释得通	讲清“为什么不是2倍”	扩展曲线 + 通信时间分解
07	数据清洗去重	自建：规则单测 + 对照组	规则是否过严（伤数据）或过松	讲清每条规则的代价	规则清单 + 有/无清洗的对照
08	SFT/DPO/RLVR	test_grpo.py 全绿 + 三方对照	是否 reward hacking、是否能力回退	逐项解释 GRPO 损失每一项	三方对照表（同口径）+ 失败样本分析
09	可验证环境	判定逻辑单测 + rollout 脚本	假阳性/假阴性样例	讲清“什么算完成”的判定	成功率 + 失败轨迹分类
10	长程 Agent	消融数据 + 轨迹日志	self-judge 是否在放大错误	讲清“什么时候 self-judge 有害”	有无 self-judge 的成功率差
11	结项	报告格式审查 (NeurIPS) + demo 可运行	结论是否超出证据范围	自我攻击 3 条 + 现场答疑	英文报告 + demo + 总复盘

四、成绩口径（对齐唐杰作业的 40% / 60%）

项	权重	内容	判据
平时作业	40%	课题 01-09 的 L1+L2+L3	每课题等权，未过者按比例扣
大项目	60%	课题 04+08+10+11 构成的整机（端到端模型 + 对照实验 + Agent 尾巴 + 报告）	问题、动机、方法、结果一项都不能糊 （唐杰原话）

注意：课程轨（CORE100299）的成绩由老师给，本表只用来自我标定；但课程大论文（70%）直接取用实训轨 W8 前的成果，所以两份成绩是同一份劳动的两个出口。

五、自欺自查清单（结项前必过）

1. 我报的每个数字，能在 **证据/** 里找到原始日志吗？
2. 我的“变好了”，有没有可能是评测口径变了 / 提示词变了 / 只挑了好种子？
3. 我有没有把**训练集上的表现**当成了能力？
4. 我写下的每个“因为…所以…”，有没有**反事实检验**（去掉这个因素会怎样）？
5. 我的图，**坐标轴和误差棒**都标了吗？有没有截断坐标轴让差距显得很大？
6. 我的结论，**最强的反方论点**是什么？写进报告了吗？

上游锁定清单（本学科只读依赖）

立单 2026-09-17。纪律：只记 commit + 用途 + 只引哪些路径，不整仓复制（仓库有体积上限）。换机器恢复的方式写在“复现命令”里。引入日期一律记当日，半年后重审一次。

一、骨架依赖（必须锁，判分与训练都靠它）

仓库	commit	引入日	许可	用途	只用哪些路径
stanford-cs336/assignment1-basics	a158843b2010	2026-09-17	MIT	课题 01/02 判分器 + handout	tests/、cs336_basics/（仅接口）、PDF
stanford-cs336/assignment2-systems	ca8bc81a59b7	2026-09-17	MIT	课题 05/06 判分器	tests/、cs336_systems/（空壳）、PDF
stanford-cs336/assignment3-scaling	03e9372992e9	2026-09-17	MIT	课题 03 规格参考（API 不可用，只读设计）	PDF、cs336_scaling/
stanford-cs336/assignment4-data	0555bea66369	2026-09-17	MIT	课题 07 过滤/去重规格	tests/、configs/、PDF
stanford-cs336/assignment5-alignment	c2734a263087	2026-09-17	△ 未声明（用前确认）	课题 08 判分器（tests/test_grpo.py）	tests/、data/、PDF
stanford-cs336/lectures	de53a9f979a6	2026-09-17	△ 未声明	讲义（lecture_01.py… lecture_17.py、PDF）	全仓可读（文本量小）
karpathy/nanochat	92d63d4e8bb4	2026-09-17	MIT	课题 04/05/06 的工程底座与对照实现	nanochat/、scripts/、runs/、tasks/、tests/

复现命令（换机器照抄）：

```
mkdir -p 上游 && cd 上游
for r in stanford-cs336/assignment1-basics stanford-cs336/assignment2-systems \
    stanford-cs336/assignment5-alignment karpathy/nanochat; do
    n=$(basename $r); git clone --depth 1 https://github.com/$r $n
done
cd assignment1-basics && git fetch --depth 1 origin a158843b2010 && git checkout a158843b2010
```

二、知识线依赖（只读，不入仓）

仓库 / 站点	commit / 版本	引入日	许可	对标模块	用法
rasbt/reasoning-from-scratch	b2e0841ee8be	2026-09-17	Apache-2.0	M6/M7/M8	教学法范本（ch3 先建评测）；ch2-4 可 CPU 跑
rasbt/LLMs-from-scratch	ace7c08802b5	2026-09-17	见仓库(NOASSERTION)	NOASSERTION	代码级最细的对照读物（附录 GQA/MoE/KV cache）

仓库 / 站点	commit / 版本	引入日	许可	对标模块	用法
huggingface/ agents-course	b3946b1d09d2	2026-09-17	Apache-2.0	M8	Agent 三框架 + 最终项目 排行榜
harbor-framework/ terminal-bench	7a337a834bff	2026-09-17	Apache-2.0	M8	长程 Agent 基准 (v2; v1 已冻结为 terminal- bench-1)
PrimeIntellect- ai/verifiers	ff3b8dc551ba	2026-09-17	MIT	M7/M8	可验证环境抽象 (课题 09 参照, 不引入代码)
jingyaogong/ minimind	7a9137d2e902	2026-09-17	Apache-2.0	M2-M6	中文术语与 RL 脚本对照 (读, 不抄进手写区)
walkinglabs/ hands-on-modern- rl	9f6e429c6894	2026-09-17	△ NOASSERTION	M6	中文 RL→RLVR 教材 (含 免费在线 Notebook)
McGill-NLP/nano- aha-moment	5314e6f8fc60	2026-09-17	MIT	M6	单文件 GRPO (阅读; 已停 更, 不作依赖)
THUDM/slime	4c193f1f3750	2026-09-17	Apache-2.0	M8/M9	智谱自家 Agentic RL 基 建 (W14 后的进阶读物)
KellerJordan/ modded-nanogpt	ecbb586296d3	2026-09-17	MIT	M4	单卡 speedrun 思路; 注 意其验证指标口径争议
datajuicer/data- juicer	1e9720d01610	2026-09-17	Apache-2.0	M4/M7	课题 07 数据清洗的规则 参考
d2l-ai/d2l-zh	e6b18ccea714 (2023-08-18)	2026-09-17	Apache-2.0	M0/M1/ M3/M5	中文地基读物; △ 仓库 2023 后基本停更, 内容仍 有效但别等更新

三、许可与合规注意

- stanford-cs336/assignment5-alignment 与 lectures 仓库未声明 SPDX 许可：只做本地学习与对照，不复制进本仓、不再分发。
- walkinglabs/hands-on-modern-rl 为 NOASSERTION：同样只读、不复制。
- 所有上游代码一律不进本仓 git（体积 + 许可双理由）；本仓只保留 commit 记录与引用的路径清单。
- 数据：只用 TinyStories（公开）、stanford-cs336/owt-sample（课程公开样本）、以及自己清洗的小语料；
△ 不要引用 HuggingFaceFW/fineweb / huggingface/fineweb（2026-09-17 实测均 404）。

四、重审机制

- 每学期期初（下一次：2027-02）重跑一遍 gh api 检查：上游是否改名/归档/换许可；
- CS336 每年改 handout（A5 已从 Qwen-2.5-Math 换为 Olmo-2-1B，新增 GSPO/MaxRL），重审时必须重新读 CHANGELOG，否则会拿着旧讲义做新作业。

课题 01 开题简报 · BPE 分词器

模块：M2 分词与表示 | 周次：W1 (09-14→09-20) | 算力：T0 纯 CPU, 0 GPU·小时 唐杰作业对应：①
从零写 Tokenizer 的前半 | 判分来源：CS336 A1 tests/test_tokenizer.py + test_train_bpe.py 手写/
代写：本课题全部核心代码属手写区 R1（见《造轮子边界.md》）

一、研究背景（这个问题历史上卡过谁）

- 2017 年前的 NLP 用整词表，未登录词（OOV）是死穴；字符级建模又让序列变得极长。
- BPE 从 2016（Sennrich 等）的机器翻译子词切分技术，被 GPT-2（2019）引入到字节级，从此成为大模型的事实标准（GPT-2/3/4、LLaMA、Qwen、GLM 都是 BPE 系）。
- 关键洞察：词表不是“词汇表”，而是一个可学习的压缩算法——它同时决定了 ①模型的输入长度 ②嵌入层参数量 ③模型能见到的最小语义单元 ④多语言/代码/数学的公平性。

二、研究问题（一句话，开放问题）

在固定词表预算下，如何设计一个训练-编码两阶段都正确、且对中文/英文/代码都公平的子词分词器？——
并回答：为什么“字节起步”是零预算下唯一不会翻车的选择？

三、攻关线索（路标，不是答案）

1. 压缩视角：如果词表大小为 V ，BPE 训练在最大化什么？把它写成“每一步减少多少 token 数”。
2. 字节起步的理由：从 Unicode 字符起步会在哪些具体输入上崩？（想 3 个真实例子再往下走）
3. 训练算法：朴素实现为什么是 $O(n^2)$ ？哪些数据结构能把它压到近似线性？（提示方向：不要用字符串拼接）
4. 并列与终止：最高频并列时怎么办？什么时候停止合并？特殊 token（`<|endoftext|>`）为什么不能参与合并？
5. 编码与解码：encode 的时候，遇到未在 merge 表里的相邻对怎么处理？为什么必须保证 `decode(encode(x)) == x`（往返一致性）？
6. 多语言公平性：同一段中文，字节级 BPE 的 token 数为什么比英文多？这对模型的“中文成本”意味着什么？

四、里程碑（可勾选，3-6 个）

- ☐ M1 手写 BPE 训练：能从语料统计词频、迭代合并、输出 merge 表与词表，并打印出前 10 个 merge 及其频次
- ☐ M2 手写编码器：支持特殊 token、未登录字节回退、往返一致性（自测 5 类边界：空串/单字节/中文/emoji/控制字符）
- ☐ M3 通过官方判分：test_tokenizer.py 与 test_train_bpe.py 全绿（原始输出贴 判卷记录.md）
- ☐ M4 压缩率对照：在 TinyStories（英）与中文小语料上各训一个 32k 词表，出压缩率表 + 与你的下注对照
- ☐ M5 改坏实验（必做）：①禁用 bytes 回退 ②不处理特殊 token ③停止条件改成“合并到只剩 1 个词”——每个先写预测，再跑，再记录现象
- ☐ M6 一页报告 + 知识卡：报告按一页四段；知识卡 ≥ 3 张进 drill-ledger

五、交付物要求

交付物	位置	验收
实现代码	手写/（你写）	通过 M3
判分原始输出	判卷记录.md	粘贴 证据/judge-*.log 原文（不许转述）
证据	证据/	run.yaml（commit/时间/命令）+ 日志 + 压缩率表 + 改坏实验记录
报告	报告.md	一页四段：问题-动机-方法-结果
知识卡	memory/drill-ledger/topics/	≥3 张（bytes 回退 / merge 规则 / 压缩率权衡）

五之二、官方判分标准（2026-09-17 实跑探明，先知道标准再动手）

判分器已接通：环境/judge/judge.sh 01。实跑探明 **28 个测试**（27 项 + 1 项 xfail），要求如下：

接口契约（你唯一需要遵守的外部形状）

```
# 手写/bpe.py 必须提供：
def train_bpe(input_path, vocab_size, special_tokens, **kwargs) -> tuple[dict[int, bytes],
list[tuple[bytes, bytes]]]
class Tokenizer:
    def __init__(self, vocab, merges, special_tokens=None): ...
    def encode(self, text: str) -> list[int]: ...
    def decode(self, ids: list[int]) -> str: ...
    def encode_iterable(self, iterable) -> Iterator[int]: ... # ← 隐藏要求，见下
```

三条硬门槛（都会挂人）

#	测试	门槛	说明
1	test_train_bpe_speed	corpus.en + vocab 500 < 1.5 秒	官方参考 0.38s；“玩具实现”约 3s 会挂
2	test_train_bpe	merges 逐项等于 参考 merges；vocab 键值集合一致	意味着预处理与 并列打破规则 都要与参考一致
3	test_encode_iterable_memory_usage	用 <code>encode_iterable</code> 流式读 5MB 语料，额外内存 ≤ 1MB （RLIMIT_AS 实测）	必须有真正的流式实现 ，不能先 read() 全部

一个官方“故意让分”的测试（别被骗）

- test_encode_memory_usage 被官方标记 **xfail**，理由原话：“Tokenizer.encode is expected to take more memory than allotted (1MB)”。→ 也就是说：**encode() 超内存是允许的，encode_iterable() 超内存是不允许的**。这个 xfail 恰好是本题“接口设计决定内存上界”的活教材（见知识地图 M5 的同类问题）。

对齐口径的隐含要求

- 与 tiktoken 的 GPT-2 逐 id 对齐 → 编码时必须**严格按 merges 的 rank 顺序**合并，而不是“最长匹配优先”；且需处理 allowed_special 与重叠特殊 token（test_overlapping_special_tokens）。
- 往返一致性覆盖：空串 / 单字符 / Unicode / 多行 / 特殊 token 尾随换行 / 地址文本 / 德语 / TinyStories。

六、开题批（开场直接发给用户，3–5 题）

见 ../memory/HANDOFF.md 的“课题 01 开题批”五题（下注题 / 概念题 / 手算题 / 权衡题 / 工程题）。**第 1 题**（中文 token 倍数）必须先存档预测，它是本学科第一条“预测→打脸”链路。

七、讲课锚点（知行合一的“讲”这一腿）

- 讲什么：M2 的五个核心问题（压缩视角 → 字节起步 → 训练算法 → 编码与往返 → 多语言成本）
- 对标材料（三师制）：
 - 主线：CS336 A1 handout 的 tokenizer 章节（英文，规格权威）
 - 中文：李沐 d2l ch14.6 子词嵌入；或 minimind 的 `train_tokenizer.py` 中文注释（只读，不得抄进手写区）
 - 补充：HF LLM Course ch2（“Implement BPE/WordPiece/Unigram from the ground up”）
- 讲完必出代码级提问（3–5 道，示例）：
 1. 把 merge 循环里“统计相邻对频次”改成“只统计出现次数 >1 的对”，词表会变得更好还是更差？预测并说明。
 2. 你的 encode 如果按“最长匹配优先”而不是“按 merge 顺序”，往返一致性还成立吗？
 3. 词表里如果混入一个重复的 token，哪一步会先崩溃？

八、参考实现（只准用于对照，不准抄）

- karpathy/nanochat 的 nanochat/tokenizer.py（接口设计与特殊 token 处理）
- rasbt/LLMs-from-scratch ch2 附带的“BPE From Scratch”
- ⚠ 纪律：GS=看接口与设计思路，不看实现细节；你的代码必须自己写。判据：M5 的改坏实验能不能被你解释——抄来的人在这里一定卡住。

判卷记录 · 课题 01 BPE 分词器

判分器：环境/judge/judge.sh 01（CS336 官方 2026 测试，锁 commit a158843b2010，test.py 未改一字）
 归档规则：只贴原始输出*，不做美化、不做转述（《判分与验收标准.md》§ 二-3）。

L1 · 基线（2026-09-17 立科时实跑）

项	结果
命令	环境/judge/judge.sh 01
结果	27 failed, 1 xfailed
失败原因	全部同一原因：FileNotFoundError：手写区还没有实现文件（接线层主动抛出的友好提示）
判分器状态	✓ 管线已验证可执行（28 个测试全部被收集并运行，非网络/环境错误）
结论	这是正确的初始状态：判分器就位，只差 手写/bpe.py

当初实跑的关键片段（完整输出会由脚本落到 证据/judge-*.log）：

```
FAILED tests/test_train_bpe.py::test_train_bpe_speed - FileNotFoundError
FAILED tests/test_train_bpe.py::test_train_bpe - FileNotFoundError
FAILED tests/test_train_bpe.py::test_train_bpe_special_tokens - FileNotFoundError
FAILED tests/test_tokenizer.py::... (24 项, 含 test_encode_iterable_memory_usage)
===== 27 failed, 1 xfailed in 3.67s =====
```

注：1 xfailed 不是缺陷——它是官方故意标记的 test_encode_memory_usage（理由：“encode 预期会超过 1MB 配额”）。详见 开题简报.md § 五之二。

L2 · 导师审稿意见

⊖ 待用户提交首次实现后填写（3-5 条：必考项 / 要记住 / 预告项）。

L3 · 费曼收口

☐ 未开始。收口动作：你对着“完全不懂分词的人”讲清 BPE，我装不懂连问三层（Why → Why not → What if），再抽 1 次“改坏实验”复述。

本课题的判分环境坑（已解决，记录备查）

坑	现象	处置
官方 conftest 需 Python ≥3.12	官方 pyproject.toml: requires-python = ">=3.12,<3.14"; conftest 用 PEP 695 泛型语法，3.11 无法解析	3.11 下自动改用等价轻量 conftest（同一 snapshot 夹具语义），并在日志头部如实声明；升到 3.12+ 后自动切回官方原文
tiktoken 需联网下载 GPT-2 词表	无外网时 SSLError 挂掉 12 项对齐测试	新增 环境/judge/prefetch_tiktoken.py: 用官方 fixture 离线构造 tiktoken 缓存(encoder.json 哈希与 tiktoken 内置期望一致，已验证 n_vocab=50257)

坑	现象	处置
PyTorch CPU 专用源不可达	download.pytorch.org SSL 被拦	bootstrap 改为：默认不装 torch（课题 01 不需要）；--with-torch 时先试 CPU 源、失败退回 PyPI

课题 02 开题简报 · Transformer 骨架与训练循环

模块: M0/M1/M3 | 周次: W2-W3 (09-21→10-04) | 算力: T0 纯 CPU, 0 GPU·小时 唐杰作业对应: ① 从零写 Transformer 的后半 | 判分来源: CS336 A1 tests/test_model.py、test_nn_utils.py、test_optimizer.py、test_serialization.py 手写: R2 (Transformer 全部组件) + R3 (训练循环与优化器) + R4 (反向传播验证)

一、研究背景

2017 年《Attention Is All You Need》删掉 RNN, 用自注意力 + 前馈层实现 **O(1) 依赖路径 + 全并行**。此后所有主流大模型都是它的变体, 但**细节已经换了一轮**: 正弦位置编码 → RoPE; LayerNorm → RMSNorm; post-norm → pre-norm; ReLU MLP → SwiGLU; MHA → GQA。本课题要求你把现代版写出来, 而不是复刻 2017 版。

二、研究问题（一句话）

一个“能从零训起来”的现代 Transformer 层, 最少需要哪几个组件? 每个组件去掉之后, 训练会以什么方式失败?

三、攻关线索

1. 形状先行: 写下每一步的张量形状 (含 batch、序列、头数), **先能在纸上跑通**, 再写代码。
2. 注意力的三个动作: 投影出 Q/K/V → 缩放点积 + 因果掩码 → softmax 加权求和并输出投影。哪一步最容易错? 为什么?
3. 位置信息: RoPE 为什么是“旋转 Q/K”而不是“加到输入上”? (讲义 M3 § 三给了恒等式)
4. 归一化与残差: 为什么 pre-norm 比 post-norm 好训? 残差提供了什么“通路”?
5. 优化器: AdamW 里“解耦权重衰减”解耦的是什么? 为什么它比 L2 正则更常用?
6. 稳定性: 梯度裁剪在什么情况下救命? fp16 vs bf16 的差别如何影响 loss 曲线?

四、里程碑

- ☐ M1 手写全部组件: 注意力 (含因果 mask 与 RoPE)、RMSNorm、SwiGLU FFN、残差堆叠、初始化
- ☐ M2 形状与数值判分通过: test_model.py / test_nn_utils.py 全绿
- ☐ M3 手写训练循环 + AdamW + LR 调度 + 梯度裁剪 → test_optimizer.py / test_serialization.py 全绿
- ☐ M4 端到端: CPU 上训一个极小模型 (d4-d8), loss 稳定下降, 并保存/加载 checkpoint 续训成功
- ☐ M5 R4 反向传播验证: 与 PyTorch autograd 的数值梯度对照 (相对误差 < 1e-6)
- ☐ M6 三组改坏实验 (去 mask / 换位置编码 / 去残差): 每组先写预测, 再跑, 再记录
- ☐ M7 一页报告 + 知识卡 ≥ 3 张

五、交付物

交付物	位置	验收
实现	手写/ (你写)	M2/M3 判分全绿

交付物	位置	验收
判分输出	判卷记录.md	证据/judge-*.log 原文
改坏实验记录	证据/改坏实验.md	预测→现象→解释 三段齐全
loss 曲线	证据/	训练日志 + 图（含坐标轴与步数）
报告	报告.md	一页四段

六、讲课锚点（讲义 M3 + 对标）

- **讲义**：讲义/M3-Transformer 内部机制.md（主干）+ 讲义/M0-数学与机器学习地基.md（§ 一 自动微分、§ 五 排查顺序）
- **对标**：CS336 lecture 03–04 · rasbt ch3–ch4 · 李沐 d2l ch10–ch11
- **必备工具链**：bash 环境/bootstrap_cpu.sh --with-torch（本课题起需要 torch）

七、代码级提问示例（讲课中出，先预测后验证）

1. 把因果 mask 从 softmax 之前挪到之后，训练现象会怎样？
2. 去掉残差后，4 层还能训、12 层完全不动——请用梯度路径解释。
3. 你把 RoPE 的最大频率底数从 10000 改成 100：短文训练会受影响吗？长上下文外推呢？
4. AdamW 的 weight_decay 若改成“直接加到梯度上”（即 L2），在大学习率下会出现什么差异？

八、风险与提醒

- ⚠ **CPU 训练极慢**：本课题只要能训起来、loss 下降即可，不追求指标（真训练在课题 04）。
- ⚠ **形状错误是最高频故障**：先写形状注释再写实现，可省一半时间。
- ⚠ **判分器需要 torch**；bootstrap --with-torch 首次约 555MB（沙箱网络受限时可能失败，届时改用 Kaggle CPU 会话）。

课题 03 开题简报 · Scaling Law 预演与打脸链路

模块：M4 | 周次：W4 (10-05→10-11) | 算力：T0 纯 CPU (极小规模 sweep) 唐杰作业对应：③ 拟合 scaling law 再外推 | 判分来源：无官方判分器 (CS336 A3 需 8 位学号 API key, 不可得) → 自建判分：预测 vs 实测的偏差 手写：R6 (拟合与外推)

一、研究背景

Kaplan 等 (2020) 与 Chinchilla (2022) 发现：loss 随参数量 N 与数据量 D 呈幂律下降，且存在一个算力最优配比。这条规律是“为什么敢花几千万训一次大模型”的全部依据。但初学者最常见的错误是：把外推当预言，不做回溯验证就把曲线画到天上去。

本课题的核心不是“拟合出一条漂亮的线”，而是建立“预测 → 验证 → 承认偏差”的科研习惯。这也是本学科“预测—打脸日志” (memory/MEMORY.md) 的主要产地。

二、研究问题（一句话）

在极小规模（我的 CPU 能承受的范围内）跑 4–5 个点，能否外推出更大模型的 loss？外推误差有多大、什么时候会崩？

三、攻关线索

- 口径：横轴用什么？参数量 N ？token 数 D ？（提示：固定预算时， N 与 D 不能同时变）
- 记账：用 M0 的 $6ND$ 估算每一次跑的算力，确保不同点是在“同样算力预算”下比较（否则曲线无意义）
- 拟合：幂律 $L = aN^{-\alpha} + c$ 的形式里，那个常数项 c 代表什么？它会不会让外推失效？
- 外推：拟合完先把预测写死在文件里（带日期），再去跑验证点 —— 顺序不能反
- 偏差来源：小规模数据是否“无限数据区”？学习率是否随规模重新调过？这些都是外推失准的技术原因

四、里程碑

- M1 设计 sweep：4–5 个规模点（如 d4/d6/d8/d10），预算固定、口径统一
- M2 跑完 sweep，记录每个点的最终 loss/bpb 与耗时（CPU 上即可，分钟级）
- M3 拟合幂律并画图（含数据点、拟合线、坐标轴标注）
- M4 先写死外推预测（例如 d12 的 bpb 预测值 + 置信区间），归档到 证据/
- M5 跑验证点，计算偏差，并解释偏差的方向与原因
- M6 一页报告：包含“预测 vs 实测”对照表（这一条是验收硬指标）

五、交付物

交付物	位置	验收
sweep 原始数据	证据/sweep.csv	含配置、loss、耗时、算力估算
拟合脚本与图	脚手架/ + 证据/	图有坐标轴、有拟合优度
外推预测存档	证据/预测-<日期>.md	跑验证点之前的文件时间戳
报告	报告.md	预测 vs 实测对照表

六、讲课锚点

- 讲义：讲义/M4-M9-待展开大纲.md 的 M4 部分 (W3 展开为完整版 M4-训练.md)

- **对标**: CS336 lecture 09/11 (scaling laws) · nanochat/runs/scaling_laws.sh 与其 miniseries 文档 · d2l 的线性回归/最小二乘 (拟合的数学基础)
- **搭桥**: probability/ 的方差与回归 —— 拟合优度、残差分析直接可用

七、代码级提问示例

1. 为什么横轴常画 **FLOPs** 而不是参数量? (提示: Chinchilla 的核心结论是什么)
2. 你拟合出的 c (常数项) 为负, 说明了什么? 还能外推吗?
3. 如果两个规模点用了不同的学习率, 曲线还能一起拟合吗? 为什么?
4. 【下注】 你预测 d12 的 bpb 会比你拟合线的预测值偏高还是偏低? 先写下答案再跑。

八、风险与提醒

- ⚠ **不要为了“好看”调学习率**: 每个点的超参策略必须写进 证据/, 否则外推无意义。
- ⚠ 极小规模会出现“学习率不敏感/过于敏感”的异常区间 —— 这本身就是很好的报告素材 (no-go 不擦除)。
- ⚠ 本课题**不花 GPU**; 如发现 CPU 太慢, 优先“缩小语料/减步数”, 不要上 Kaggle (配额留给课题 04)。

课题 04 开题简报 · 0.1B 端到端预训练（首次上机）

模块: M4/M5 | 周次: W5-W6 (10-12→10-25) | 算力: T2 Kaggle 2×T4, 计划 12-21 GPU·小时 唐杰作业对应: ① 端到端训一个 0.1B | 判分来源: 自建 (loss/bpb 达标 + 采样可读性) + nanochat/core_eval.py 的 CORE 手写: 复用课题 01/02 的产物 (tokenizer + 模型 + 优化器), 本课题新增数据管线与训练编排的判断

一、研究背景

“0.1B”是唐杰作业里的数字,也是单卡/双卡免费算力下的上限。参考点: nanochat 的 d12 约等于 GPT-1 量级 (官方讨论区原话: “d12 是我最喜欢的模型,它是 GPT-1 的大小,约 6 分钟训完”——那是在 8×H100 上)。本课题的核心不是刷指标,而是把“训练一次完整模型”的每个环节亲自走通并记账。

二、研究问题（一句话）

在 2×T4、20 GPU·小时的预算内,我能不能从零训出一个“能生成连贯英文、并在 CORE 上有非零分”的 0.1B 模型? 实测成本与估算差多少?

三、攻关线索

- 1. 数据: TinyStories (全量) + owt-sample 采样。先算一遍: 词表大小 × 语料字节 → token 数 → 需要的步数
- 2. 规模选择: nanochat 用单一旋钮 --depth; 在 T4 上 16GB 显存能放多深? (用 M0 的显存公式先估)
- 3. 精度: T4 是 SM75, 没有 bf16 → 必须 fp16 + GradScaler (或用 nanochat 的 NANOCHAT_DTYPE=float16)
- 4. 断点续训: Kaggle 会话 9-12h 会断 → checkpoint 必须写 /kaggle/working (20GB 持久)
- 5. 超时自停: 脚本内置 wall-clock 上限, 防止忘记关会话
- 6. 指标: 训练看 bpb (词表无关), 最终跑 CORE (22 数据集合成, core_eval.py 单文件)

四、里程碑

- ☐ M1 实验卡获批: 写清假设/命令/预算上限/停止条件/成功判据 (见《算力与成本预算.md》§ 五)
- ☐ M2 会话预检: nvidia-smi 确认 2×T4 (若是 P100 立即重开会话)
- ☐ M3 冒烟: --depth=4、100 步, 确认能跑、能存、能续训 (≤10 分钟)
- ☐ M4 首次正式训练: 双 T4 + DDP, 跑到预算上限或 loss 平台
- ☐ M5 续训一次 (验证 checkpoint 真的可用 —— 这一条最容易翻车)
- ☐ M6 评测: bpb + CORE + 采样 20 条样本 (人工判断 “是否连贯”)
- ☐ M7 成本账单: 实测耗时 vs 估算 (M0 的 6ND + T4 折算), 逐项解释偏差
- ☐ M8 一页报告 + 知识卡 (含 “我最意外的三个现象”)

五、交付物

交付物	位置	验收
实验卡	证据/实验卡-NN.md	经用户确认
run.yaml + 日志	证据/	含 commit、配置、随机种子、耗时
曲线	证据/	loss/bpb vs FLOPs 与 vs 时间 两张
采样样本	证据/样本.md	≥20 条, 标注 “通顺/不通顺”

交付物	位置	验收
成本账单	报告.md	实测 vs 估算偏差 $\leq 2\times$ 或给出解释
checkpoint	Kaggle 持久盘	不进程（体积原因）；只在 证据/ 留哈希与路径

六、讲课锚点

- 讲义：M4-训练.md（W3 展开）
- 对标：nanochat/README.md 与 runs/speedrun.sh（读它怎么组织全流程）· CS336 lecture 05/13-14
- 搭桥：课题 03 的 scaling 结论在这里第一次被“真跑”检验

七、代码级提问示例

1. 双 T4 跑 d12，为何不是 2 倍加速？先预测扩展效率（50%？80%？），再测。
2. T4 上 fp16 与 fp32，谁更快？谁更稳？GradScaler 在解决什么？
3. 你的 loss 在第 2000 步开始上升。按 M0 § 五 的顺序，你的前三个动作是什么？
4. 若把语料从 TinyStories 换成中文语料（词表不变），loss 会怎么变？bpb 呢？

八、风险与提醒

- **！ P100 陷阱**：本课题如果拿到 P100，训练慢 3-5 倍且后续课题 05 直接不可用 → 宁可等 T4 时段。
- **！ 配额**：本课题占全学期约 1/5 的配额，实验卡必须写实；冒烟不通过不许开正式训练。
- ⚠ 0.1B 在 T4 上大概率需要“降 batch + 梯度累积”，这会改变有效 batch size → 要写进 run.yaml。
- ⚠ Kaggle 会话到期会杀进程：每 500 步存一次是硬要求。

课题 05 开题简报 · 手写 Triton attention kernel

模块：M5 | 周次：W7-W8 (10-26→11-08) | 算力：T1 Kaggle 1×T4，计划 10 GPU·小时 唐杰作业对应：② 手写 Triton attention kernel，测多卡训练与推理增益 | 判分来源：CS336 A2（正确性对齐）+ 自建性能对照 手写：R5（kernel 本体：分块 + online softmax）

一、研究背景

朴素注意力要把 $L \times L$ 的注意力矩阵整个写进显存： $L = 8192$ 时单头就是 64M 个元素。FlashAttention (2022) 的核心洞察：**根本不需要把矩阵写出来**——分块计算、在线维护 softmax 的最大值与归一化因子，就能得到数学上完全等价的结果。它把注意力的显存从 $O(L^2)$ 降到 $O(L)$ 。

Triton 让这件事对非 CUDA 专家也可做：用 Python 风格的 DSL 写“块级”程序。本课题的目标不是写出最快的 kernel，而是亲手经历“数值等价 + 显存下降 + 速度提升”这三件事的因果链。

二、研究问题（一句话）

一个分块 + online softmax 的手写 kernel，能否在数值上等价于朴素实现，并把显存从 $O(L^2)$ 降到 $O(L)$ 、速度提升若干倍？提升来自哪里（算术减少？访存减少？）

三、攻关线索

1. 先量化瓶颈：朴素实现在 $L=2k/8k$ 时的显存、耗时（用 profiler 看是 compute-bound 还是 memory-bound）
2. online softmax 的递推：维护 running max m 与 running sum ℓ ，新块的贡献要按 $e^{m_{\text{old}} - m_{\text{new}}}$ 缩放——这正是 M0 §四 logsumexp 的应用
3. 分块维度：按 query 分块、沿 key 方向扫描 → 每个块只需常驻 SRAM
4. 数值容差：与 PyTorch SDPA 对照，相对误差应在 $1e-2 \sim 1e-3$ (fp16)；先写容差，再判定
5. 多卡/推理增益：训练侧测吞吐 (tokens/s)，推理侧测 decode 显存与延迟——这是唐杰作业②要求的“增益”

四、里程碑

- □ M1 基线：朴素注意力的显存与耗时 ($L=2k/4k/8k$ 三点)
- □ M2 正确性：手写 kernel 与 SDPA 的数值对照 (max abs error 记录在案)
- □ M3 显存：证明峰值显存随 L 近似线性（对比朴素实现的 $O(L^2)$ ）
- □ M4 速度：给出 speedup 曲线（含 L 与 block size 两个维度）
- □ M5 profiling：用 profiler 说明“提升来自哪里”（访存 vs 算术，给出数字）
- □ M6 增益测量：训练吞吐 (tokens/s) 与推理（融合进 attention 后）的实测对比
- □ M7 一页报告 + 知识卡（online softmax 递推必须能徒手推）

五、交付物

交付物	位置	验收
kernel	手写/（你写）	M2 数值对照通过
三张图	证据/	speedup / 显存 / profile（坐标轴与硬件口径必须标注）

交付物	位置	验收
数值误差记录	证据/	含容差设定与依据
报告	报告.md	必须回答“提升来自哪里”

六、讲课锚点

- **讲义**：讲义/M5-推理.md (W5 展开) 中“GPU 内存层级与 Triton 编程模型”一节
- **对标**：CS336 lecture 05–06 (GPUs / Kernels & Triton) + A2 handout (**flash-attn 需两步安装**：
uv sync --no-install-package flash-attn 再 uv sync)· Triton 官方 FlashAttention 教程
- **搭桥**：M0 § 四的 logsumexp —— 你会亲手把它写进 kernel

七、代码级提问示例

1. online softmax 里，为什么新块要乘 $e^{m_{\text{old}} - m_{\text{new}}}$ ？不乘会怎样？
2. block size 从 64 增到 256：显存、速度、数值误差各往哪走？
3. 为什么 L 很大时，朴素实现容易 OOM 而 kernel 不会？(用 $O(L^2)$ vs $O(L)$ 说清)
4. 【下注】你预测 speedup 在 L=2k 时是多少倍？先写，再测。

八、风险与提醒

- **！ P100 不可用于本课题**：Triton 要求计算能力 ≥ 7.0 ，P100 是 6.0 $\rightarrow$ 开跑前必须验明是 T4。
- $\triangle$ 正确性优先于速度：**数值对照不过，速度数字一律不算数**。
- $\triangle$ 硬件口径陷阱：任何 speedup 数字必须写明“对比对象 (PyTorch SDPA / 朴素实现)”与“硬件 (T4)”，否则曲线毫无意义（这一条写进报告硬要求）。
- $\triangle$ 若 T4 上 kernel 因共享内存不够而失败，**先降 block size，不要放弃**（这本身就是教学素材）。

课题 06 开题简报 · 多卡与分布式训练

模块: M4 | 周次: W9 (11-09→11-15) | 算力: T2 Kaggle 2×T4, 计划 8 GPU·小时 唐杰作业对应: ② 的“多卡训练增益”部分 | 判分来源: CS336 A2 分布式部分 + 自建 benchmark 手写: R3 的延伸 (并行策略选择与通信原语判断)

一、研究背景

数据并行 (DDP) 把同一模型复制到每张卡、各自算一个微批、再**同步梯度**; 当模型放不下单卡时, 才有 FSDP/ZeRO (切分参数、梯度、优化器状态)、张量并行 (切层内矩阵)、流水线并行 (切层)。Kaggle 免费档给的是 2×T4, 且走 PCIe 而非 NVLink —— 这恰好是“观察通信成为瓶颈”的最佳实验台。

二、研究问题

在弱互联 (PCIe) 的 2×T4 上, DDP 的扩展效率是多少? 瓶颈是计算还是通信? FSDP 在什么规模才开始划算?

三、攻关线索

1. 先对齐正确性: DDP 后的梯度应与单卡 (同等效批大小) **数值一致**, 先验证再谈性能
2. 批大小语义: 2 卡 × 每卡 batch = 等效批大小; 若显存受限要**同步调整学习率** (线性/平方根缩放)
3. 计时口径: step time 要区分“纯计算时间”与“含 all-reduce 的时间”; 用 torch.profiler 或手写计时
4. 扩展效率: $E = \frac{T_1}{2T_2}$ 。先**预测** (PCIe 下 0.6–0.8?) 再测
5. 通信占比: 梯度同步量与参数量成正比 (2N 字节/步, all-reduce 近似); 序列长/批小时通信占比更高
6. FSDP 的代价: 切分带来额外通信 (all-gather + reduce-scatter), **小模型上往往更慢**

四、里程碑

- ☐ M1 DDP 梯度一致性验证 (与单卡同等效批) → 记录最大误差
- ☐ M2 单卡 vs 双卡的 step time、tokens/s 对照 (同模型、同等效批)
- ☐ M3 **扩展效率曲线** (至少 3 个配置点: 不同 batch/序列长度)
- ☐ M4 **通信占比分解**: 计算 / all-reduce / 数据加载 三段计时
- ☐ M5 一次 FSDP 实验并解释“为什么 (不) 划算”
- ☐ M6 **先写预测再实测**: 预测扩展效率, 跑完对照 (进 memory/MEMORY.md 打脸日志)
- ☐ M7 一页报告

五、交付物

交付物	位置	验收
分布式脚本	手写 / 或 脚手架 / (按 B3 分类)	可复跑
计时原始数据	证据/bench.csv	含每配置 3 次重复
扩展效率曲线	证据/	标注硬件 (2×T4 PCIe) 与对比基准
报告	报告.md	解释“为什么不是 2 倍”

六、讲课锚点

- 讲义: M4-训练.md (W3 展开) 中并行一节

- **对标:** CS336 lecture 07-08 (Parallelism) · nanochat/optim.py (分布式优化器实现) · HF Ultra-Scale Playbook (用前先核)
- **搭桥:** M0 的“梯度累加” → DDP 就是跨卡的梯度累加

七、代码级提问示例

1. $2 \times T4$ 上 DDP 只快了 1.4 倍，列出三个最可能的原因。
2. 把每卡 batch 减半、用梯度累积补回等效批，通信量会怎么变？
3. 什么情况下 FSDP 会比 DDP 慢？（用通信量解释）
4. all-reduce 的通信量是多少字节/步？用参数量 N 表示。

八、风险与提醒

- ⚠ Kaggle 双 T4 会话**不稳定**（可能只给一张）：预检时确认 `torch.cuda.device_count()==2`。
- ⚠ 不要在本课题纠缠“调到最快”——目标是理解扩展效率的成因，配额要留给课题 08。

课题 07 开题简报 · 数据清洗与去重

模块：M7/M4 | 周次：W10 (11-16→11-22) | 算力：T1 Kaggle 1×T4，计划 6 GPU·小时 唐杰作业对应：③ 从 raw dump 洗语料 | 判分来源：CS336 A4 (过滤/去重规格) + 自建消融 手写：R8 (过滤/去重的判定规则设计)

一、研究背景

“数据决定上限”已是共识：C4/FineWeb/Dolma 等工作的主要贡献不是模型而是数据管线。典型手段：语言识别、质量启发式（重复度、符号比、平均行长）、有毒内容过滤、近似去重（MinHash/LSH）。最有教学价值的现象：**清洗规则过严会伤害模型**（把有价值的长尾数据也删了）。

二、研究问题

在固定算力下，“清洗 + 去重”能把同样规模模型的下游表现提升多少？每条规则的收益与代价各是多少？

三、攻关线索

1. **基线优先**：先训一个“未清洗”的基线（可与课题 04 的结果复用）—— **没有对照就无法声称“清洗有效”**
2. **规则设计**：至少三条可解释规则（如：短行比例、符号-词比、n-gram 重复率），每条要能说清“它在拦什么”
3. **去重量级**：精确去重（哈希）与近似去重（MinHash + LSH）的成本差异；文档级 vs 段落级
4. **消融**：逐条关掉规则各跑一次（小规模即可），得“每条规则的边际贡献”表
5. **口径**：所有对照必须同 **token 预算**、**同分词器**、**同评测**（否则结论无效）
6. **常用工具对照**：datajuicer/data-juicer (Apache-2.0, 活跃) 与 allenai/dolma 的规则集可作参考

四、里程碑

- ☐ M1 数据来源与规模确定（用 owt-sample / TinyStories 扩样，或小规模 CC 采样）
- ☐ M2 手写 3 条启发式规则 + 1 个去重算法（精确哈希起步，有余力再上 MinHash）
- ☐ M3 规则单元测试（含**假阳性样例**：被误删的好数据长什么样）
- ☐ M4 训练对照：未清洗 vs 全部清洗 vs 逐条消融（≥4 组跑）
- ☐ M5 结果表：每组的下游指标 + **数据保留率**（这是被忽略的关键列）
- ☐ M6 一页报告：必须回答“哪条规则最值钱、哪条可能有害”

五、交付物

交付物	位置	验收
过滤/去重实现	手写/	M3 单测通过
规则清单与理由	报告.md	每条规则说明“拦什么、代价是什么”
对照实验数据	证据/对照.csv	含保留率与指标
假阳性样例	证据/	≥5 条被误删的好数据

六、讲课锚点

- **讲义**：M7-评测.md (W9 展开) 中“数据质量与评测的关系”
- **对标**：CS336 lecture 13-14 (Data) + A4 handout · datajuicer/data-juicer 规则文档 · Dolma 论文
- **搭桥**：概率论的采样与分布 —— 过滤本质是“改变训练分布”，会引入选择偏差

七、代码级提问示例

1. 过滤掉“短行”能提分，但会不会系统性删掉对话/代码类数据？怎么检测这种偏差？
2. 精确去重 vs 近似去重：各自漏掉什么、误杀什么？
3. 数据保留率从 90% 降到 50%，你如何判断“这是提纯还是阉割”？
4. 【下注】预测“全部清洗”相对“未清洗”的提升幅度，先写再测。

八、风险与提醒

- ⚠ 磁盘：本课题可能涉及较大语料，**大文件绝不进 git**（见学科 .gitignore）。
- ⚠ 不要一开始就上 MinHash 集群版：**先精确去重跑通闭环**，再考虑近似。
- ⚠ 4 组对照训练会吃掉配额：**用小模型 + 短序列**，保证口径一致即可。

课题 08 开题简报 · SFT / DPO / RLVR 三方对照

模块: M6/M7 | 周次: W11-W12 (11-23→12-06) | 算力: T2 Kaggle 2×T4, 计划 18 GPU·小时 (全学期最贵) 唐杰作业对应: ④ 同一基座上把 SFT、DPO、RLVR 做对照 | 判分来源: CS336 A5 tests/test_grpo.py + 自建三方对照 手写: R7 (损失函数与优势估计)

一、研究背景

预训练给“知识”，后训练给“行为”。三条主流路线：

方法	需要什么	优化什么
SFT	(指令, 回答) 对	最大似然 (只对回答部分算 loss)
DPO	(指令, 好回答, 坏回答) 偏好对	隐式奖励差 (无需显式 reward model)
RLVR	可判对错的答案 + 验证器	期望可验证奖励 (GRPO 用组内相对优势)

本课题的教学价值极高：它是课程大纲“机器学习（强化学习）”与唐杰作业④的交点，也是“知识讲解”最能发挥的地方——GRPO 的每一项都能在概率论里找到根（期望、方差、条件期望）。

二、研究问题

同一个基座、同一套评测，SFT / DPO / RLVR 各自把什么能力提升了、把什么能力损害了？训练奖励的上升，有多少是真的能力提升？

三、攻关线索

- 基座与数据**：基座用课题 04 的模型（或 0.1B 级开源小模型）；任务是可验证的（数学题/格式约束/单元测试）
- SFT 的损失掩码**：只对回答部分算 loss，指令部分不算——这是个高频错误点
- DPO 的推导**：为什么 $\sigma\left(\beta \log \frac{\pi(y_w)}{\pi_{ref}(y_w)} - \beta \log \frac{\pi(y_l)}{\pi_{ref}(y_l)}\right)$ 能等价于“隐式奖励”？
- GRPO 的四步**：采样一组输出 G 个 → 打分 → 组内归一化得优势 $A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$ → 加权策略梯度 + KL 惩罚
- KL 惩罚的作用**：防止策略跑离参考模型太远（对应 RLHF Book 的“过度优化”章）
- 评测口径**：训练奖励 ≠ 能力。必须在 held-out 题集上评，并且给误差棒
- 制造一次 reward hacking**：例如让模型学到“输出格式对了但答案是瞎猜”，记录现象（这是本课题最好的素材）

四、里程碑

- M1 SFT 基线（含损失掩码正确性验证：打印一条样本的 mask）
- M2 DPO 跑通 + 与 SFT 的对照（held-out）
- M3 手写 GRPO 损失并逐项注释（优势从哪来、为什么要除标准差）
- M4 tests/test_grpo.py 通过（判分器接入后）
- M5 三方对照表（同基座、同评测、同种子策略，带误差棒）
- M6 reward hacking 实验：故意设计一个“可被钻空子”的奖励，观察并记录
- M7 一页报告 + 知识卡（GRPO 递推必须能徒手推）

五、交付物

交付物	位置	验收
三个训练脚本	手写/（损失与优势估计属 R7）	可复跑
判分输出	判卷记录.md	test_grpo.py 原文
对照表	证据/	含误差棒与评测口径说明
reward hacking 记录	证据/	现象 + 机制解释
报告	报告.md	“能力提升 vs 训练奖励” 必须分开讨论

六、讲课锚点

- 讲义：M6-后训练.md（W10 展开）
- 对标：rasbt《Reasoning from Scratch》ch3（先建评测）+ ch6–ch7（GRPO）·《RLHF Book》ch4/6/8/14·CS336 A5 + lecture 15–17·minimind/trainer/train_grpo.py（中文工程实现）·nano-aha-moment（单文件）
- 搭桥（重点）：probability/ 的期望、方差、条件期望——GRPO 的优势归一化就是“标准化”，KL 惩罚就是“分布差异”，REINFORCE 就是“用样本均值估期望”

七、代码级提问示例

1. 为什么 SFT 只对回答部分算 loss？若指令部分也算，会发生什么？
2. GRPO 里为什么要除以组内标准差？如果一组所有奖励完全相同，优势是多少？这时梯度会怎样？
3. DPO 不需要 reward model，那它的“奖励”藏在哪儿？（用对数比写出来）
4. 训练奖励从 0.3 涨到 0.8，held-out 准确率却从 40% 掉到 35%。列出三个可能原因。
- 5.【下注】预测三方在 held-out 上的排序，先写再测。

八、风险与提醒

- **！最贵的一周**：18 GPU·小时是全学期 1/5，三个方法各只跑一次正式实验，其余用小规模调试。
- **！不做 held-out 评测就等于自欺**：训练奖励必须与独立评测同时报（写进报告硬要求）。
- ⚠ A5 官方仓库未声明 SPDX 许可 → 只做本地学习对照，不复制进仓（见《上游锁定清单》§ 三）。
- ⚠ Kaggle 会话上限 9–12h：RL 训练务必支持断点续训，否则一次会话跑不完。

课题 09 开题简报 · 可验证环境 + harness

模块：M7/M8 | 周次：W13 (12-07→12-13) | 算力：T2 Kaggle 2×T4, 计划 6 GPU·小时 唐杰作业对应：⑤ 搭可验证环境 + harness | 判分来源：自建 (判定逻辑单测 + rollout 成功率) 手写：R9 (判定逻辑 / verifier)

一、研究背景

2026 年的关键词是“可验证奖励”：与其训练一个会拍马屁的奖励模型，不如造一个能判定对错的环境。CS336 没有这一环，Berkeley Agentic AI MOOC 专门有一讲叫“Post-Training Verifiable Agents”，PrimeIntellect-ai/verifiers 把它抽象成库。唐杰作业⑤要求“搭可验证环境 + harness”正是这个方向。“可验证”的三要素：①任务是可判定的（不是主观好坏）②环境能给出稳定反馈 ③harness 能重复运行并复现。

二、研究问题

我能为一个小程序设计一个“可自动判定”的任务环境吗？判定逻辑的假阳性/假阴性有多高？模型在这个环境里的成功率是多少？

三、攻关线索

- 1. 选题：从三个里选一个（按成本从低到高）
 - ①数学题解答（用数值比对 + 容错；MATH-500 风格）
 - ②函数补全（隐藏单元测试；本质是迷你 HumanEval）
 - ③格式约束生成（如严格 JSON / 表格填充，规则判定）
- 2. 判定逻辑的三大坑：等价但不相同（1/2 vs 0.5）、解析失败当错、边界用例漏判
- 3. 假阳性检测：人为构造“错误但能骗过判定器”的答案 → 测出假阳性率（这是质量的核心指标）
- 4. harness 设计：任务装载 → 模型生成 → 判定 → 记录轨迹 → 复现（固定种子）
- 5. 可重复性：同一批任务跑两次，成功率波动多大？（误差棒思维，M7 讲过）
- 6. 对照：可验证奖励 vs 规则弱判定（如只看格式），用同一模型比成功率差异

四、里程碑

- ☐ M1 选定任务类型，写 30–100 道题的题集（含标准答案与判定规则）
- ☐ M2 判定逻辑单测：等价形式、边界、异常输入（≥10 个用例）
- ☐ M3 假阳性实验：构造 10 个“错误但可能蒙混”的答案，测出漏判率
- ☐ M4 harness 跑通：给基座模型跑一遍，得成功率 + 失败样本
- ☐ M5 失败分类：把失败归成 3–5 类（格式错 / 计算错 / 中途放弃 / 判定器误杀）
- ☐ M6 一页报告：判定器质量 + 模型表现 两个都要报

五、交付物

交付物	位置	验收
题集与判定器	手写 / （判定逻辑 R9）	M2 单测通过
harness 脚本	脚手架 / （B3）	固定种子可复跑
失败样本	证据 /	≥20 条，分类标注

交付物	位置	验收
假阳性报告	报告.md	含漏判率与典型案例

六、讲课锚点

- **讲义**: M8-智能体.md (W13 展开)
- **对标**: Berkeley Agentic AI MOOC 的 “Post-Training Verifiable Agents” 与 “Agent Evaluation” 讲 · PrimeIntellect-ai/verifiers (只读概念, 不引入代码) · harbor-framework/terminal-bench 的任务设计思路 · **rasbt ch3** (评测器设计: 解析 + verifier + 基准式检查)

七、代码级提问示例

1. 你的判定器把 1/2 判错、把 0.50 判对——这属于哪一类错误? 如何系统性避免?
2. 假阳性率高会导致 RL 训练出现什么后果? (联系课题 08 的 reward hacking)
3. 同一批任务两次运行成功率差 8%。你会先怀疑模型还是 harness? 怎么验证?
4. 为什么“用 LLM 当裁判”在可验证任务上通常是**退步**?

八、风险与提醒

- ⚠ **判定器本身也要被评测**: 只报模型成功率不报判定器质量, 是不完整的实验。
- ⚠ 题集不要抄公开 benchmark (污染问题), **自己出题 + 自己判定**才是本课题的价值。
- ⚠ 配额有限: 优先“题集质量高、任务简单”, 不要一上来做代码执行沙箱 (那是课题 10 的事)。

课题 10 开题简报 · 长程 Agent 与 self-judge

模块: M8 | 周次: W14 (12-14→12-20) | 算力: T2 Kaggle 2×T4, 计划 8 GPU·小时 唐杰作业对应: ⑤
训长程 Agent, 鼓励 self-judge loop | 判分来源: 自建 (消融实验 + 轨迹日志) 手写: R10 (self-judge 协议设计)

一、研究背景

从“回答问题”到“完成任务”的跨越, 靠的是 **harness (外壳)**: 工具调用、上下文管理、失败重试、自我评估。Karpathy 与李宏毅都专讲 **context engineering**; nanochat 里 `execution.py` 就是一个最小的“能执行代码的 Agent 外壳”。本课题的落点是**长程任务**: 多步串联后误差累积, 以及 **self-judge (自我评判)** 到底有没有用。

二、研究问题

多步任务中, 失败主要来自模型能力还是来自 harness 设计? 加上 self-judge loop 后成功率提升多少——还是反而下降?

三、攻关线索

1. 最小 ReAct: 不用任何框架, 手写 Thought → Action → Observation 循环 (三四十行就能跑)
2. 工具设计: 先给两个工具 (计算器 / 受限代码执行)。工具接口的清晰度往往比模型智力更决定成败
3. 长程任务构造: 把课题 09 的题改造成“需要 3-5 步”的复合任务 (如: 先算 → 再验 → 再格式化)
4. 失败分类: 格式错 / 工具调错 / 目标漂移 (忘了原始要求) / 循环打转 / 提前放弃
5. self-judge 的两种形态:
 - 自评重试: 模型给自己打分, 低于阈值就重做 (rasbt ch5 的 self-refinement)
 - 自评过滤: 生成多个候选, 自己挑最好的 (best-of-N 的变体)
6. 关键实验 (消融): 同样的任务集, 三组对照 = 无 self-judge / 自评重试 / 自评过滤
7. 警惕: self-judge 会放大原有错误 (模型判不出自己错在哪, 反而确信) → 必须留失败案例

四、里程碑

- ☐ M1 手写最小 ReAct 循环 (无框架), 跑通单步任务
- ☐ M2 加入第二个工具 + 多步任务集 (≥30 个任务)
- ☐ M3 轨迹记录规范: 每步的 thought/action/observation 落盘 (可复现)
- ☐ M4 基线成功率 + 失败五分类 (每类给 2 个真实案例)
- ☐ M5 self-judge 消融: 三组对照, 给出成功率差与误差棒
- ☐ M6 反例收集: 找出 ≥3 个“self-judge 导致更差”的案例并解释机制
- ☐ M7 一页报告 + 知识卡

五、交付物

交付物	位置	验收
ReAct 循环	手写/	可跑通多步任务
self-judge 协议	手写/ (R10)	有明确的判定时机与阈值设计说明
轨迹日志	证据/trajectories/	每任务一份, 含步数与结果

交付物	位置	验收
消融数据	证据/	三组成功率 + 误差棒
报告	报告.md	必须回答“何时 self-judge 有害”

六、讲课锚点

- 讲义：M8-智能体.md (W13 展开)
- 对标：李宏毅《Context Engineering》专讲 (Agent 时代的关键技术) · Berkeley LLM Agents MOOC (ReAct 原始论文那讲) · HF Agents Course (三框架对照 + 观测性/评测 bonus unit) · **rasbt ch5 (self-refinement)** · nanochat/execution.py+tests/test_execution.py (**沙箱执行的最小实现**, 可读接口不可抄实现)
- 搭桥：M5 的 KV cache (长上下文的显存代价) → 为什么长程 Agent 必须做上下文压缩

七、代码级提问示例

1. 长程任务失败率随步数指数上升。**分解**：是每步错误率问题，还是错误会累积传播？
2. self-judge 什么时候**一定**无效？(提示：当模型无法区分“看起来对”和“真的对”时)
3. 上下文压缩 (截断/摘要) 会引入什么新失败模式？
4. 为什么“工具返回值要结构化”能显著提升成功率？(从 token 分布角度解释)
5. 【下注】预测 self-judge 在简单任务 vs 困难任务上的效果差异，先写再测。

八、风险与提醒

- **！ 不要用框架起步**：手写一遍 ReAct 再去看 LangGraph/smolagents, 否则学不到 harness 的本质。
- ⚠ self-judge 会显著增加 token 消耗与时间 → 算力预算要写进实验卡。
- ⚠ 轨迹日志可能很大 → 只保留结构化摘要 (thought 截断 + 结果), 避免污染仓库体积。

课题 11 开题简报 · 结项 (NeurIPS 报告 + 现场 demo)

模块: M9 | 周次: W15-W16 (12-21→2027-01-03) | 算力: 弹性 ≤ 4 GPU·小时 唐杰作业对应: ⑥ 英文 NeurIPS 格式 + W16 现场 demo、⑦ 作业 40% / 大项目 60% | 判分来源: 报告协议 + 导师审稿 + 自我攻击

一、研究背景

唐杰的原话是“问题、动机、方法、结果，一项都不能糊”。这恰好也是本学科九份章程反复强调的：没有证据的结论不采纳、no-go 不擦除、误差棒必填。结项不是“写一篇总结”，而是把 11 个课题的证据链组装成一份可被审稿的作品。

二、研究问题 (结项版)

我能不能用一份英文技术报告 + 一个可运行的 demo，证明我独立完成了大模型全链路，并且知道自己的结论在哪里不成立？

三、攻关线索

1. 报告骨架 (NeurIPS 风格, 8-10 页): Abstract / Introduction / Related Work / Method / Experiments / Results / Discussion & Limitations / Conclusion
2. 只写有证据的结论: 每个数字都能指回 证据/ 中的原始日志
3. Limitations 必须诚实: 规模受限 (0.1B vs 前沿)、算力受限 (免费档)、评测口径的局限
4. Demo 设计: 现场演示要可在 3 分钟内跑完 (预生成 checkpoint, 不现场训练)
5. 自我攻击 3 条: 主动写出“我的结论最可能错在哪”
6. 一稿多用: 这份报告同时作为课程大论文 (70%) 与唐杰作业⑦的交付物

四、里程碑

- ☐ M1 证据清点: 11 个课题的 证据/ 全部齐备, 缺的补齐 (这是一次“审计”)
- ☐ M2 英文报告初稿 (Abstract + Method + Experiments 先写)
- ☐ M3 图表规范化: 所有图有坐标轴、单位、误差棒、样本数
- ☐ M4 自我攻击 3 条 + 导师审稿 3-5 条 → 修订
- ☐ M5 Demo 准备: 脚本 + 预生成模型 + 3 分钟演练
- ☐ M6 总复盘: 写进 memory/LEARNINGS.md (哪些讲法有效、哪些坑最贵)
- ☐ M7 课程大论文定稿并提交 (署名直填: 孙承泽 / 2253710052 / 能动强基 2501)

五、交付物

交付物	位置	验收
英文报告	课题 11-结项/report-en.md (+ PDF)	格式合规、结论有据
中文大论文	课程轨/大论文/	课程要求格式
Demo	课题11-结项/脚手架/demo.sh	3 分钟内可跑
复盘	memory/LEARNINGS.md + HANDOFF.md	可被下一任直接接续

六、讲课锚点

- 讲义: M9-前沿.md (W15 展开)

- 对标: rasbt ch3 的“先建评测”精神贯穿全文 · CS336 报告规范 · 论文精读/ 的英文写作规范
- 搭桥: turbine-blade-ai-platform (应用章节); 人工智能基础与应用/课程轨/大论文 (一稿两用)

七、自检清单 (结项前必过, 抄自《判分与验收标准》§ 五)

1. 每个数字都能在 证据/ 找到原始日志吗?
2. “变好了” 有没有可能是口径变了 / 只挑了好种子?
3. 有没有把**训练集表现**当能力?
4. 每个“因为…所以…” 有没有反事实检验?
5. 图表的坐标轴与误差棒都标了吗?
6. 我写下的最强反方论点是什么?

八、风险与提醒

- ⚠ 报告最易犯的错是**夸大**: 规模受限就要说受限, 这反而是可信度来源。
- ⚠ 不要为了“好看”删掉 no-go 记录——用户明确要求“负结果不许擦除”。
- ⚠ W16 现场 demo 必须有**失败预案** (预生成结果 + 录屏), 不依赖现场网络与算力。